



M A S T E R T H E S I S

In Order to Obtain the

PROFESSIONAL MASTER
in
S T A T I S T I C S

Presented and defended by

Israa Saad

October 2026

Evaluating Feature Selection Across Predictive Models for E-Commerce Fraud Detection

Supervisor
Dr. Abbas Rammal

Reviewers
Dr. Hicham Abdallah
Dr. Zainab Assaghir

Lebanese University-Faculty of sciences

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Dr. Abbas Rammal, for his support and guidance through this project. I also extend my appreciation to everyone who contributed to the completion of this study. Also, I thank the professors in the Department of Applied Mathematics and Statistics at the Lebanese University for the knowledge and skills I gained throughout my studies. Their courses and guidance contributed greatly to my academic development and provided the foundation needed to complete this research.

I am especially grateful to my family for their continuous support, patience and encouragement throughout my Master's studies. Their presence and understanding helped me continue working through the demanding periods of this journey.

Abstract

The growth of e-commerce has increased the importance of detecting fraudulent transactions in large, high-dimensional and imbalanced datasets. Feature selection is commonly used to reduce dimensionality, but its benefits should be evaluated against the complete feature space and across different classifiers. This study examines whether feature selection can reduce dimensionality while maintaining predictive performance, and how its effect depends on the classifier.

Using the IEEE-CIS Fraud Detection dataset (590,540 transactions, 433 predictors, 3.5% fraud), seven feature-selection methods covering filter, embedded, wrapper and metaheuristic approaches, each retaining 120 predictors, were compared with a no-selection baseline across six classifiers (48 experiments). The data were split chronologically, with preprocessing and selection restricted to the training data, and class imbalance was handled through cost-sensitive learning. PR-AUC was the primary evaluation metric and ROC-AUC a secondary metric, and differences between methods were assessed using Friedman tests with Shaffer-corrected comparisons. Decision thresholds for six configurations were also selected using chronological validation data.

RFE with Random Forest achieved the highest individual PR-AUC (0.5523), while RFE with XGBoost achieved the highest ROC-AUC (0.9112). Across classifiers, the no-selection condition achieved the highest mean PR-AUC (0.4425), but RFE retained 99.2% of this performance, with no statistically significant difference between the two conditions under either metric. The effect of selection depended on the classifier: mutual information improved Logistic Regression PR-AUC but substantially reduced Random Forest performance. Metaheuristic searches required considerably more computation than RFE without achieving higher average held-out performance. The models also achieved high ROC-AUC and accuracy but more moderate PR-AUC and MCC values, highlighting the importance of metric choice in imbalanced fraud detection. In the supplementary threshold analysis, the MCC of RFE with Random Forest increased from 0.451 to 0.540 using a validation-selected threshold.

Overall, RFE produced the strongest individual configurations under both metrics while using 72.3% fewer predictors and much less feature-selection time than the metaheuristic methods. Although no selection method improved mean PR-AUC over the complete feature space across the six classifiers, these results show that substantial dimensionality reduction can be achieved with very little loss in predictive performance. The effect of feature selection, however, depended on the classifier used.

Keywords: e-commerce fraud detection, feature selection, recursive feature elimination, high-dimensional data, class imbalance, machine learning, PR-AUC, ROC-AUC, IEEE-CIS.

Table of Contents

Acknowledgements **1**
Abstract **2**
Table of Contents **3**
List of Tables **5**
List of Figures **6**
Chapter 1. Introduction **7**
1.1 Study Background and Motivation 7
1.2 Statement of the Problem 9
1.3 Research Objectives 9
1.4 Scope and Contributions of the Study 10
Chapter 2. Literature Review **11**
2.1 Fraud Detection and Class Imbalance 11
2.1.1 Fraud Detection as a Classification Problem 11
2.1.2 Class imbalance and the choice of evaluation metric 11
2.1.3 Imbalance-handling strategies 12
2.2 Feature selection 12
2.2.1 Types of feature selection 12
2.2.2 Metaheuristic feature selection 12
2.2.3 Filter, Embedded and Wrapper Methods 14
2.2.4 Feature selection in the fraud domain 15
2.3 Classification algorithms 16
2.4 Statistical comparison of multiple methods 17
2.5 The research gap 18
Chapter 3. Methodology **20**
3.1 Experimental Design 20
3.2 Data and Preprocessing 21
3.2.1 The Dataset and Feature Composition 21
3.2.2 Preprocessing Procedure 22
3.2.3 Resulting partitions 23
3.3 Feature Selection Methods 23
3.3.1 Common subset size 24
3.3.2 Filter and embedded methods 24
3.3.3 Recursive feature elimination 24
3.3.4 Metaheuristic search methods 25
3.3.5 Summary of conditions 26
3.4 Predictive Models 26
3.4.1 The classifiers 27
3.4.2 Model configurations 28
3.5 Class Imbalance 28
3.6 Evaluation Metrics 29
3.6.1 Validation-Based Threshold Selection 30
3.7 Statistical Validation 30
3.7.1 Friedman and Iman–Davenport tests 30
3.7.2 Shaffer post-hoc comparisons 30
3.7.3 Feature-subset similarity 31
3.8 Software and Reproducibility 31
3.9 Experimental Validity 31
Chapter 4. Results **33**
4.1 Overall Predictive Results 33
4.1.1 PR-AUC Across the 48 Experiments 33

4.1.2 Discrimination under ROC-AUC and PR-AUC 34
4.1.3 Threshold-Dependent Performance 36
4.2 Feature-Selection Outcomes 38
4.2.1 Overall Performance by Selection Condition 38
4.2.2 Metaheuristic Search Behavior 39
4.2.3 Overlap Between Selected Feature Subsets 39
4.2.4 Composition of the Selected Subsets 40
4.3 Classifier Performance 41
4.4 Statistical Validation 41
4.4.1 Feature-Selection Conditions 42
4.4.2 Classifiers 43
4.5 Practical Performance-Parsimony-Cost Comparison 43
4.6 Summary of Findings 44
Chapter 5. Discussion and Conclusion 45
5.1 Interpretation of the Main Findings 45
5.1.1 Feature Reduction and Predictive Performance 45
5.1.2 Differences Between Feature-Selection Methods 46
5.1.3 Interaction with Classifier Choice 46
5.1.4 Performance, Parsimony and Computational Cost 47
5.1.5 Interpreting the Performance Level 47
5.2 Practical Implications for Feature Selection 48
5.3 Relation to Previous Research 49
5.4 Scope and Boundaries of the Findings 49
5.5 Recommendations and Directions for Future Work 51
5.6 Conclusion 51
References 53
Appendix A. Exploratory Data Analysis 55
Appendix B. Full Experimental Results 57
B.1 Performance of all 48 experiments 57
B.2 Confusion-matrix counts 58
B.3 The selected feature subsets 60
B.4 Validation-based threshold selection 61
B.5 Accuracy of the 48 experiments 63
B.6 Native missing-value and categorical handling 64
Appendix C. Shaffer Post-hoc Comparisons 65
C.1 Feature-selection conditions, PR-AUC 65
C.2 Classifiers, PR-AUC 66
C.3 Feature-selection conditions, ROC-AUC 67
C.4 Classifiers, ROC-AUC 68
Appendix D. Preprocessing Implementation 69

List of Tables

Table 1.1. Characteristics of the IEEE-CIS Dataset. 8
Table 2.1. Feature selection studies in the fraud domain 15
Table 3.1. Predictor families in the merged dataset 22
Table 3.2. Chronological partitions. 23
Table 3.3. Settings of the metaheuristic searches. 25
Table 3.4. Summary of the eight feature-selection conditions. 26
Table 3.5. Classifier configurations, held constant across all eight conditions. 28
Table 4.1. Mean ROC-AUC and mean PR-AUC by classifier, averaged over the eight selection conditions 35
Table 4.2. Mean ROC-AUC and mean PR-AUC by selection condition, averaged over the six classifiers 35
Table 4.3. Mean threshold-dependent metrics at probability threshold 0.50. 36
Table 4.4. Test-set classification performance at the default and validation-selected thresholds. 37
Table 4.5. Performance across the six classifiers by feature-selection condition. 38
Table 4.6. Internal search fitness and final held-out performance of the metaheuristic selectors. 39
Table 4.7. Feature-family counts in the selected subsets 40
Table 4.8. Classifier performance across the eight feature-selection conditions 41
Table 4.9. Friedman and Iman–Davenport tests on PR-AUC and ROC-AUC. 42
Table 4.10. Performance retention and feature-selection cost relative to the no-selection control 43
Table B.1. Held-out performance of every feature-selection and classifier combination. 57
Table B.2. Confusion-matrix counts for every combination at threshold 0.50 58
Table B.4. Validation-set metrics across decision thresholds for the six configurations. 61
Table B.5. Test-set accuracy at threshold 0.50 for the 48 feature-selection–classifier combinations 63
Table B.6. PR-AUC and ROC-AUC of the boosting classifiers (and RFE + RF) with the original preprocessing and with native handling of missing values and categorical variables. 64
Table C.1. All 28 pairwise comparisons among the eight feature-selection conditions, on PR-AUC. 65
Table C.2. All 15 pairwise comparisons among the six classifiers, on PR-AUC. 66
Table C.3. All 28 pairwise comparisons among the eight feature-selection conditions, on ROC-AUC. 67
Table C.4. All 15 pairwise comparisons among the six classifiers, on ROC-AUC. 68

List of Figures

Figure 1.1. The thirty columns with the highest missing-value proportions; the dashed line indicates the 95% threshold. 8
Figure 3.1. The experimental pipeline. 21
Figure 4.1. PR-AUC across the 48 feature-selection-classifier experiments 33
Figure 4.2. Ten highest-performing feature-selection-classifier combinations by PR-AUC 34
Figure 4.3. Recorded GA, PSO and GWO search trajectories using internal validation ROC-AUC. 39
Figure 4.4. Jaccard similarity between the seven selected feature subsets. 40
Figure A.1. Correlation among the V1–V339 engineered variables. 55
Figure A.2. Distribution of absolute pairwise correlation among the V variables (mean |r| = 0.153; 2.6% of pairs > 0.90) 55
Figure A.3. Correlation among the count variables C1–C14. Most pairs are strongly positively correlated. 56
Figure A.4. Fraud rate by derived transaction hour (left) and by day of week (right). 56

Chapter 1. Introduction

1.1 Study Background and Motivation

The rapid growth of e-commerce and digital payments has increased the number and speed of online transactions. At the same time, payment fraud remains an important challenge for businesses and financial institutions. Fraudulent transactions may involve stolen payment information, compromised accounts or false identities, leading to financial losses, chargebacks and problems for legitimate customers.

Fraud detection systems often combine rule-based controls with machine learning methods. While rules identify transactions that meet predefined indicators of suspicious activity, supervised machine-learning models learn patterns from labelled transaction data to estimate fraud probabilities. These models can capture relationships among multiple characteristics that may be difficult to represent through individual rules.

Fraud data, however, present several modelling challenges. One is class imbalance, since fraudulent transactions usually represent only a small proportion of all observations. Under this imbalance, accuracy alone can be misleading because a model may correctly classify most legitimate transactions while detecting relatively few frauds. Evaluation therefore requires measures that better reflect performance on the minority fraud class.

High dimensionality creates another challenge. Fraud datasets may contain large numbers of variables describing transactions, payment characteristics, customer behavior, devices and identity information. Some variables may provide useful complementary information, whereas others may be redundant, weakly informative or noisy. As the number of predictors increases, computational requirements and model complexity may also increase without a corresponding improvement in predictive performance.

These challenges are particularly relevant to the IEEE-CIS dataset used in this study. The merged transaction and identity data contain hundreds of variables from different feature families, including transaction, payment-card, address, email, device, identity and anonymized engineered variables by Vesta Corporation. After preparation, 433 predictors are available for modelling.

Table 1.1. Characteristics of the IEEE-CIS Dataset.

Characteristic	Observed value
Transactions	590,540
Fraudulent transactions	20,663 (3.5%)
Identity records	144,233
Predictors after preparation	433
V1–V339 variables	339
Predictors with >80% missing values	74
Predictors with >95% missing values	9
Mean absolute correlation in V block	0.153
V-feature pairs with |r| > 0.90	2.6%
PCs required for 95% V-block variance	104

The feature space is also highly heterogeneous in terms of data availability. Identity information is recorded for only 144,233 of the 590,540 transactions, resulting in extensive missingness after the transaction and identity tables are merged. Overall, 74 predictors have more than 80% missing values, including nine identity variables with more than 95% missingness and several exceeding 99%. Highly incomplete predictors were retained for empirical evaluation rather than assumed to be uninformative, because the observed and missing portions may still contain useful information. Figure 1.1 illustrates the extent of missingness among the 30 most incomplete predictors.

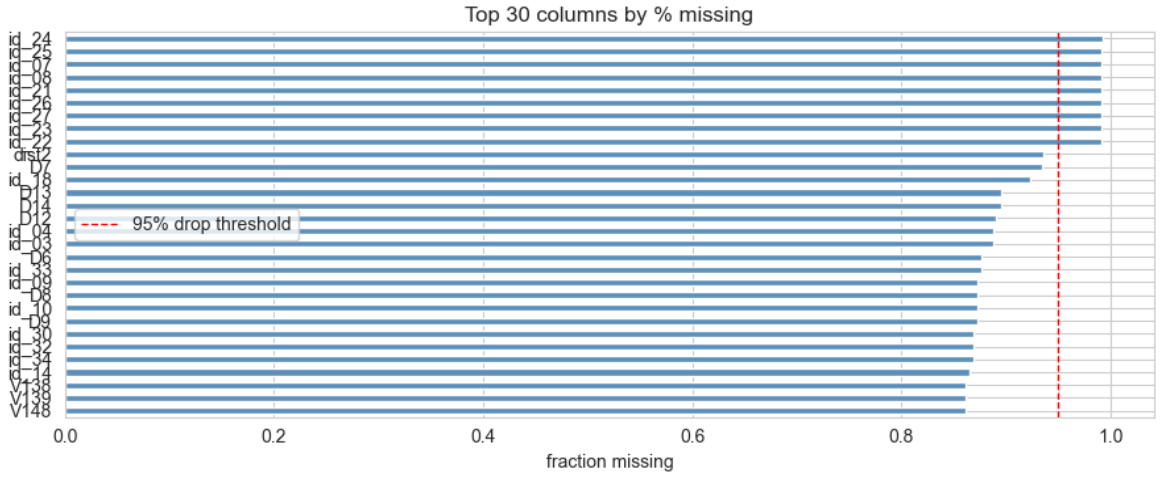


Figure 1.1. The thirty columns with the highest missing-value proportions; the dashed line indicates the 95% threshold.

Exploratory analysis also indicates that redundancy is present but is not uniform across the feature space. Strong correlations occur within some feature groups; for example, several of the C1–C14 count variables contain overlapping information. However, the much larger V1–

V339 block, which represents most of the predictor space, shows a more complex structure. Its mean absolute pairwise correlation is 0.153, only 2.6% of computable feature pairs have absolute correlations above 0.90, and 104 principal components are required to explain 95% of its variance. These findings indicate that strong redundancy is confined to pockets, while substantial variation remains spread across many dimensions. Consequently, reducing the feature space may remove redundant or weakly informative predictors, but it may also discard complementary information. This makes the benefit of feature selection an empirical question rather than an assumption.

Additional exploratory analyses of the temporal structure of the data and of the correlation structure among the engineered variables are provided in Appendix A.

Feature selection aims to reduce dimensionality by retaining informative predictors, potentially improving computational efficiency and model simplicity. However, its effect on predictive performance depends on the data, selection method and classifier. This motivates comparing different feature-selection approaches with the complete feature space across several predictive models.

1.2 Statement of the Problem

High-dimensional fraud datasets raise an important modelling question: whether to use all available predictors or only a selected subset. Retaining all available predictors preserves the information contained in the original feature space, but it can increase model complexity and computational requirements. Feature selection reduces dimensionality, yet removing predictors does not guarantee better fraud detection. Variables that appear weak individually may still contribute useful information when combined with other predictors, meaning that feature reduction can remove relevant information as well as redundancy. This decision becomes more challenging in an imbalanced setting, where fraudulent transactions represent only a small proportion of the observations.

The effectiveness of feature selection also depends on the predictive model used afterward. Different selection methods identify useful predictors in different ways, while classifiers differ in how they use the selected information. A feature subset that performs well with one classifier could perform considerably worse with another. The conclusion may also depend on the evaluation measure, particularly in an imbalanced classification problem where ROC-AUC and PR-AUC provide different views of model performance. Comparing feature-selection methods with only one classifier, or without a complete-feature baseline, could provide an incomplete picture of their value.

The central problem addressed in this study is therefore whether explicit feature selection provides a meaningful advantage over using the complete feature space for high-dimensional e-commerce fraud detection. The study investigates this problem using the IEEE-CIS Fraud Detection dataset.

1.3 Research Objectives

The main objective of this study is to evaluate whether feature selection can reduce data dimensionality while maintaining or improving predictive performance in high-dimensional e-commerce fraud detection. Rather than assuming that reducing the number of predictors will

lead to better model performance, the study compares feature selection with the complete feature space and examines whether its effect changes across different predictive models.

To achieve this objective, seven feature-selection methods are examined, covering filter, embedded, wrapper and metaheuristic approaches. The selected feature subsets are evaluated across six classifiers, with a no-feature-selection condition serving as the baseline. This design provides a consistent basis for comparing the selection methods across classifiers.

The study also examines the selected feature subsets beyond predictive performance alone. The methods are compared in terms of the similarity and composition of the predictors they retain, the amount of dimensionality reduction achieved and the computational cost of feature selection. Predictive performance is evaluated primarily using PR-AUC because of the imbalanced nature of the fraud data. ROC-AUC is used as a complementary measure to examine whether the findings are consistent across different evaluation criteria. In addition, a supplementary analysis uses chronological validation data to select decision thresholds for representative models, allowing precision and recall to be evaluated beyond the default threshold of 0.5. Statistical comparisons are also used to assess whether the observed differences among feature-selection methods and classifiers are supported across the experimental conditions.

1.4 Scope and Contributions of the Study

This study focuses on feature selection for high-dimensional e-commerce fraud detection using the IEEE-CIS Fraud Detection dataset, which contains 590,540 transactions and, after preparation, 433 predictors with a fraud rate of approximately 3.5%. The analysis covers seven feature-selection methods and six classifiers. To ensure a consistent comparison, each method selects a common budget of 120 predictors, while the no-selection condition retains all 433. The resulting eight feature conditions are evaluated across the six classifiers, giving 48 experiments in total. Classifier settings are fixed across feature conditions and are not tuned separately for each subset. The study therefore isolates the effect of changing the feature set under controlled modelling conditions.

The main contribution of the study is a systematic comparison of feature-selection methods with the complete feature space as an explicit baseline, rather than assuming that dimensionality reduction is necessarily beneficial. The experimental design also preserves the temporal structure of the data through a chronological train-test split. Preprocessing and feature selection use the training data only, reducing the risk that information from the later test period influences model development. In addition, evaluating each feature-selection method across several classifiers makes it possible to examine whether its effect depends on the predictive model. The analysis also considers whether the findings differ when ROC-AUC is used instead of PR-AUC. A supplementary validation-based threshold analysis further examines the precision–recall trade-off for selected model configurations.

Finally, the study considers predictive performance alongside dimensionality reduction, differences among selected feature subsets and the computational cost of selection. Together, these elements provide a broader assessment of the value and trade-offs of feature selection in high-dimensional fraud detection.

Chapter 2. Literature Review

This chapter reviews the literature underlying the thesis in four main areas: fraud detection and class imbalance, feature-selection methods, classification algorithms used to evaluate the selected subsets, and statistical methods for comparing competing approaches. The review examines relevant methods, findings and limitations in previous studies to establish the theoretical background for the methodology and identify the research gaps addressed in this thesis.

2.1 Fraud Detection and Class Imbalance

2.1.1 Fraud Detection as a Classification Problem

Payment fraud detection is a binary classification problem on tabular transaction data, where the objective is to distinguish a relatively small number of fraudulent transactions from a much larger number of legitimate ones. Dastidar, Caelen and Granitzer [1] survey the machine learning literature on this problem and describe supervised tree ensembles as widely used approaches in the literature, applied to transaction-level features.

The IEEE-CIS dataset used in this thesis reflects these characteristics. Diaconu and Pele [2] report 590,540 transactions, including 20,663 fraudulent cases, corresponding to a fraud rate of approximately 3.5%, across 432 features. This class distribution illustrates the imbalance faced in the present study and provides a naturally imbalanced setting for evaluating fraud-detection models.

2.1.2 Class imbalance and the choice of evaluation metric

Accuracy can be highly misleading in fraud detection due to the small proportion of fraudulent transactions. In IEEE-CIS, predicting all transactions as legitimate would still produce an accuracy of 96.5%. On the ULB credit card dataset used in many fraud detection studies, where the fraud rate is only 0.172%, the same trivial classifier would achieve 99.83% accuracy. Accuracy values close to these baseline levels are reported in parts of the fraud detection literature.

ROC-AUC is commonly used as an alternative to accuracy, but it also has limitations in highly imbalanced settings. Davis and Goadrich [3] show that optimizing ROC-AUC does not necessarily optimize precision–recall performance. Saito and Rehmsmeier [4] further explain that ROC curves can be misleading when the negative class is dominant, as a large number of false positives may still correspond to a low false-positive rate. Precision–recall analysis therefore provides a more informative view of performance on the rare fraud class and the effect of false positives.

This distinction is evident in Alamri and Ykhlef [5], who report a ROC-AUC of 98.14% but a PR-AUC of only 67.70% for the same model on approximately six million simulated PaySim transactions, demonstrating how metric choice can substantially affect the interpretation of performance.

One additional measure is relevant to this study. The Matthews correlation coefficient (MCC) is recommended for imbalanced binary classification by Boughorbel, Jarray and El-Anbari [6]. Mienye and Sun [7] instead use the geometric mean of sensitivity and specificity as a wrapper fitness measure, but this measure is not used in this study.

2.1.3 Imbalance-handling strategies

Synthetic oversampling is used in a large share of the fraud literature, including Zhao and Bai [8], Saputra and Suharjito [9] and Esenogho et al. [10]. Its implementation, however, requires care. Tran, Zhang and Xue [11] show that generating synthetic observations before splitting the data can allow information from the test set to influence the training data, and therefore recommend partitioning the data first. Dal Pozzolo et al. [12] also show that resampling can affect predicted probabilities, preserving their ranking but changing their calibration.

Cost-sensitive learning provides an alternative to modifying the class distribution through resampling. Instead of generating or removing observations, it assigns greater importance to errors on the minority class during model training. This allows the original observations and class distribution to be retained while increasing the influence of fraudulent transactions on the learning process [43].

These approaches show that imbalance handling is closely connected to the experimental design. Resampling requires careful ordering to avoid information leakage and may affect predicted probabilities, while cost-sensitive learning provides an alternative that retains the original observations and class distribution.

2.2 Feature selection

2.2.1 Types of feature selection

Feature selection aims to identify a smaller subset of potentially useful predictors, reducing dimensionality and potentially improving predictive performance, computational efficiency and model parsimony. Al-Tashi et al. [13] classify feature selection methods into three main categories: filter methods, which rank features using statistical criteria independently of a classifier; wrapper methods, which evaluate candidate subsets according to classifier performance; and embedded methods, which perform feature selection as part of model training. While filters are generally computationally efficient, wrappers can account for interactions between the feature subset and the learning algorithm at a considerably higher computational cost.

For search-based feature selection, candidate subsets are commonly represented as binary vectors, where 1 indicates a selected feature and 0 an excluded feature [13]. The effectiveness of feature selection may also depend on the classifier: tree-based models can perform an internal form of feature selection and are relatively robust to irrelevant variables [36], whereas other learning algorithms can be more sensitive to high-dimensional inputs [14]. Therefore, the benefit of feature selection should be evaluated across different classifiers rather than assumed in advance.

2.2.2 Metaheuristic feature selection

Why heuristic search

Finding an optimal feature subset becomes difficult as the number of features increases, since n features produce 2^n possible subsets. Exhaustive evaluation is therefore impractical for high-dimensional data. Metaheuristic algorithms address this problem by searching for promising feature subsets iteratively without evaluating every possible combination.

Agrawal et al. [15] reviewed more than 100 metaheuristic algorithms published between 2009 and 2019, grouping them into evolutionary, swarm intelligence, physics-based, and human-

based methods. They found that wrapper studies rely mainly on k-nearest neighbors (42%) and support vector machines (29%), while the common objective function gives greater weight to classification accuracy (0.99) than to the number of selected features (0.01).

Genetic algorithms

A genetic algorithm for feature selection encodes candidate feature subsets as binary chromosomes, where each bit indicates whether a feature is included or excluded. It evolves a population of these subsets across generations, scoring each by model performance, selecting the fittest as parents, then producing offspring through crossover and mutation. This continues until a stopping condition is met.

Ileberi, Sun and Wang [16] in their study applied a genetic algorithm to feature selection for credit card fraud detection. They used a random forest inside the fitness function and took its classification accuracy as the fitness value.

Mienye and Sun [7] provide another example of using a genetic algorithm for feature selection on real credit card data. Their approach is hybrid: information gain is first used to reduce the feature set, followed by a genetic algorithm wrapper using an extreme learning machine as the internal evaluator. The fitness function is based on the geometric mean of sensitivity and specificity, which the authors choose because accuracy is considered unsuitable for imbalanced classification.

Particle swarm optimization

Particle swarm optimization treats each candidate feature subset as a particle moving through the search space, guided by its own best position and the swarm's global best. In its typical form, a fitness function evaluates each subset, for example via cross-validation, and particles update their velocity and position accordingly. This continues iteratively until a stopping condition is reached.

Tran, Zhang and Xue [11] proposed a hybrid method that integrates filter and wrapper selection within a single process. When a particle finds a new personal best, a local search based on symmetric uncertainty removes selected features that are more strongly associated with other features than with the class label and adds non-selected features with above-average association with the class. Their fitness function combines balanced accuracy, chosen to account for class imbalance, with a distance term. The authors also highlight two practical considerations: performing feature selection on the entire dataset can introduce selection bias, requiring the test set to remain independent of preprocessing and feature selection, while greedy stepwise backward selection proved computationally prohibitive and failed to complete within a week on their datasets. These findings support training-only preprocessing and feature selection, as well as the need for computationally efficient feature-selection strategies.

The grey wolf optimizer

The grey wolf optimizer mimics wolf pack hierarchy, where the three best feature subsets (alpha, beta, delta) guide the rest of the population's position updates. A wrapper-based fitness function commonly evaluates each candidate binary subset using classifier accuracy and feature count. The population iterates toward convergence by repositioning relative to these three leaders until a stopping condition is met.

Emary, Zawbaa and Hassanien [17] produced the first binary versions of the algorithm for feature selection. They proposed two approaches: one uses a stochastic crossover of the alpha,

beta and delta positions, and the other applies a sigmoid transfer function to convert continuous positions into binary values.

More recently, Stephan et al. [18] compared the original GWO with eight variants across ten high-dimensional datasets under a common experimental framework. The original GWO remained highly competitive, ranking second overall and outperforming six of the eight variants, while the results revealed trade-offs between dimensionality reduction, computational efficiency, and subset stability. However, the study used only one wrapper classifier, did not conduct parameter sensitivity analysis, and focused mainly on small-sample biomedical and image datasets, leaving the performance of GWO on large-scale industrial and transactional data relatively unexplored.

2.2.3 Filter, Embedded and Wrapper Methods

Alongside metaheuristic approaches, filter, wrapper and embedded methods provide alternative strategies for reducing the feature space. The methods most relevant to the present study include mutual information, LASSO, random-forest importance and recursive feature elimination.

Mutual information. Mienye and Sun [7] use information gain as the filter stage of their hybrid feature selection method, noting its equivalence to mutual information. Lee, Park and Kim [20] combine information gain with correlation-based selection and the Pearson correlation coefficient, taking the intersection of the three, and reduce ninety features to ten. Siam et al. [19] combine it with correlation filtering and random-forest importance. These studies illustrate the computational simplicity of information-based filtering, although ranking predictors individually may overlook information that emerges through interactions among variables.

Recursive feature elimination. Guyon et al. [21] introduced RFE as an iterative method that repeatedly trains a model and removes the least important feature or group of features. For high-dimensional data, they suggest eliminating several features at each step to reduce computational cost, although this produces rankings of feature subsets rather than individual variables. They also show that recursive elimination and a single importance ranking can produce different subsets, supporting their evaluation as distinct feature selection approaches.

Priscilla and Prabha [22] apply RFE to the IEEE-CIS dataset using XGBoost as the base estimator, after first reducing the feature space with mutual information. Their approach therefore represents a two-stage feature selection process combining filter and wrapper methods. They report that this selection increased the ROC-AUC of XGBoost from 79.8% to 85.5%.

Darst, Malecki and Engelman [23] highlight an important limitation of RFE. Comparing recursive elimination with random forest importance on genomic data, they find that RFE can remove redundant correlated variables but may also eliminate important variables under strong correlation. This suggests caution when applying RFE to high-dimensional datasets with correlated features.

Random forest importance. Wang, Liang, Hancock and Khoshgoftaar [24] compare ten importance-based feature selection methods, five based on SHAP and five on built-in model importance across five classifiers and different subset sizes on the highly imbalanced ULB fraud dataset. Using PR-AUC with repeated five-fold cross-validation, they find that larger feature subsets generally outperform smaller ones and that random forest ranks highest

among the tested classifiers. However, feature selection was performed on the entire dataset before cross-validation, which may introduce selection bias; therefore, their reported performance values are not directly adopted here. The study also excludes the time variable because its near-unique values may allow the model to memorize observations.

LASSO. Tibshirani [25] introduced L1 regularization, while Ng [14] discusses its application to feature selection with logistic regression. Friedman, Hastie and Tibshirani [26] describe the regularization path and emphasize predictor standardization, unlike tree-based methods that generally do not require scaling. Under strong correlation, LASSO tends to select one predictor from a group of correlated variables while excluding others [27], an important consideration for high-dimensional datasets containing correlated feature groups.

In fraud detection, El Barakaz Fatima et al. [28] apply an elastic-net sparse penalty as embedded feature selection across four credit datasets. Their study reports that sparse selection without resampling underperformed the no-selection baseline across all four datasets and both classifiers. However, their numerical results are not used here because resampling and feature selection were conducted before data partitioning, and some reported geometric means appear inconsistent with the corresponding recall values.

2.2.4 Feature selection in the fraud domain

The methodological literature on feature selection is broad, but its application to fraud detection is more limited and is concentrated on a relatively small number of datasets. Table 2.1 summarizes the main studies identified in this review that apply feature selection to fraud-related data.

Table 2.1. Feature selection studies in the fraud domain

Study	Feature selection method	Dataset
Ileberi, Sun and Wang [16]	Genetic algorithm wrapper	ULB credit card
Mienye and Sun [7]	Information gain + genetic algorithm wrapper	ULB credit card
Mniai, Tarik and Jebari [29]	Filter, wrapper and embedded methods	Credit card
Wang, Liang, Hancock and Khoshgoftaar [24]	SHAP-based and importance-based ranking	ULB credit card
Siam, Bhowmik and Uddin [19]	Correlation + information gain + random forest importance	Credit card
Priscilla and Prabha [22]	Mutual information + XGBoost recursive feature elimination	IEEE-CIS
Lee, Park and Kim [20]	Information gain + correlation-based selection + Pearson correlation	Crowdfunding
Ravisankar et al. [30]	Feature selection	Financial statement fraud
El Barakaz Fatima et al. [28]	Elastic-net sparse selection	Four credit datasets

A notable pattern in the reviewed literature is the frequent use of the ULB credit card dataset, which contains 28 PCA-transformed variables together with transaction time and amount, without missing or categorical values. Its structure differs substantially from IEEE-CIS, which contains a larger and more heterogeneous feature space with missing values, categorical variables, and groups of related features. Among the reviewed studies, Priscilla and Prabha [22] apply feature selection directly to IEEE-CIS by combining mutual information with XGBoost-based RFE, while Diaconu and Pele [2] use the dataset to evaluate predictive classifiers without focusing primarily on feature selection. Overall, comparatively limited attention has been given to evaluating different feature selection approaches on IEEE-CIS under the same experimental framework.

PSO and GWO feature selection studies, in contrast, commonly rely on gene-expression, molecular, and image datasets with relatively few observations but thousands of features, where high dimensionality primarily reflects a large number of variables relative to sample size. IEEE-CIS presents a different setting, with 590,540 transactions and several hundred features. Lee et al. [31] also use real-world data, but their positive class represents approximately 39.5% of observations, compared with about 3.5% fraud in IEEE-CIS. These differences suggest that conclusions obtained from smaller, transformed or less imbalanced feature spaces may not transfer directly to the large and heterogeneous IEEE-CIS setting.

2.3 Classification algorithms

Classifiers play two roles in this study. XGBoost is used as the evaluator within RFE and the three metaheuristic searches, contributing to the selection of feature subsets. All six classifiers are then trained on each feature-selection condition to evaluate its predictive performance. Classifier choice is therefore relevant not only to prediction itself, but also to determining whether the usefulness of a selected subset is consistent across different learning mechanisms.

Logistic regression provides a linear baseline. The model was introduced by Cox [32] and relates a linear combination of the predictors to the probability of a binary outcome. Each predictor contributes through an estimated coefficient, making the model directly dependent on the representation of the input feature space. Logistic regression is also sensitive to predictor scaling, while correlated predictors can make individual coefficient estimates unstable. Ng [14] examines logistic regression in the presence of many irrelevant predictors and shows how regularization and feature selection become relevant in high-dimensional settings. Logistic regression therefore provides a useful contrast to the more flexible non-linear models considered in this study.

The multilayer perceptron (MLP) provides a non-linear alternative to linear and tree-based models. Information passes from the input layer through one or more hidden layers, where non-linear activation functions allow the network to represent more complex relationships among predictors. Training adjusts network weights through backpropagation [33], typically using cross-entropy loss for binary classification, while regularization can be used to control overfitting [34]. Unlike tree-based models, MLPs generally require attention to input scaling, and their performance can be sensitive to the representation of the input features and model configuration.

Random Forest was introduced by Breiman [35] as an ensemble of decision trees constructed using bootstrap samples and random subsets of predictors at each split. Each tree partitions the feature space through successive decision rules, and predictions are aggregated across the ensemble. Random feature sampling reduces dependence among trees, while aggregation

helps reduce the variance associated with individual decision trees. Because each split is chosen from a random subset of predictors, some uninformative predictors may have limited influence on the fitted model. However, the composition of the available feature space still affects which predictors are considered as candidates at each split.

Gradient boosting constructs learners sequentially, with each new learner added to improve the current ensemble according to a specified loss function [36]. The three implementations used in this study differ in how they extend this general procedure. XGBoost incorporates regularization, shrinkage, column subsampling and computational techniques for efficient learning, including the handling of sparse inputs [37]. LightGBM uses histogram-based learning and leaf-wise tree growth to improve computational efficiency on large datasets [38]. CatBoost introduces ordered target statistics and related techniques designed to improve the treatment of categorical information and reduce prediction shift [39]. XGBoost is particularly relevant to the present study because it serves both as one of the final predictive classifiers and as the evaluator used within RFE and the metaheuristic feature-selection methods.

The different learning mechanisms of these classifiers are relevant to feature selection. Linear and neural-network models combine information across their input variables, whereas tree-based models choose predictors when constructing individual splits. Some uninformative predictors may therefore have limited influence on tree-based models during fitting, although the available feature space can still affect the resulting model. External feature selection may consequently affect different classifier families in different ways. Grinsztajn, Oyallon and Varoquaux [40] find that tree-based models generally outperform neural networks across their tabular benchmarks and identify robustness to uninformative features as one important difference between the model families. Their results, however, were obtained within a particular benchmark and extensive hyperparameter-search setting and should not be interpreted as establishing that tree-based models are universally superior.

Existing evidence also indicates that the effect of changing the available feature space is not necessarily consistent across datasets. Chen and Guestrin [37] report that column subsampling improved performance on one dataset but reduced it on another, illustrating that reducing the available predictors does not guarantee better prediction. For IEEE-CIS specifically, Diaconu and Pele [2] benchmark XGBoost and CatBoost against a heterogeneous graph neural network and report stronger performance for the gradient-boosting models in their experiments.

Taken together, these differences support evaluating feature selection across classifiers with different learning mechanisms rather than assuming that a subset which benefits one classifier will provide the same advantage to another.

2.4 Statistical comparison of multiple methods

A study that compares several methods across multiple conditions requires statistical testing to determine whether observed performance differences are systematic rather than due to variation across the evaluation conditions. Demšar [41] presents a commonly used non-parametric framework for comparing multiple learning algorithms based on ranks. The methods are ranked separately within each evaluation block, with rank 1 assigned to the best-performing method. The average rank of method j across n blocks is

$$\bar{R}_j = \frac{1}{n} \sum_{i=1}^{n} r_{ij}$$

where r_{ij} is the rank of method j in block i.

The Friedman test uses these ranks to test the null hypothesis that the compared methods have equal expected performance. For k methods evaluated across n blocks, the Friedman statistic can be written as

$$\chi^2_F = \frac{12n}{k(k+1)} \left[\sum R_{j^2} - \frac{k(k+1)^2}{4} \right]$$

Because the original Friedman statistic can be conservative when the number of evaluation blocks is limited, the Iman–Davenport correction transforms it into an F-distributed statistic:

$$F_F = \frac{(n-1)\chi^2_F}{n(k-1) - \chi^2_F}$$

Rejecting the omnibus null hypothesis indicates that at least one method differs from the others, but it does not identify which pairs are responsible for the difference. Post-hoc comparisons are therefore required.

García and Herrera [42] discuss multiple-comparison procedures for this setting. For two methods i and j, the difference between their average ranks can be standardized as

$$Z_{ij} = \frac{\bar{R}_i - \bar{R}_j}{\sqrt{[\, k(k+1) / (6n) \,]}}$$

When several pairwise hypotheses are tested simultaneously, adjustment is required to control the family-wise error rate. A simple Bonferroni adjustment can be conservative because it treats all pairwise hypotheses independently. Shaffer's static step-down procedure uses the logical relationships among the hypotheses to provide greater statistical power while maintaining family-wise error control [42]. The hypotheses are ordered according to their unadjusted p-values and evaluated sequentially using the allowable numbers of simultaneously true hypotheses.

Kendall's coefficient of concordance, W, can additionally be used to describe the degree of agreement among the rankings. Values closer to 1 indicate stronger agreement among the blocks, whereas values closer to 0 indicate weaker agreement. Together, the omnibus test, post-hoc comparisons and rank-based summaries provide a framework for assessing whether apparent differences among learning methods are statistically supported rather than relying only on average predictive performance.

2.5 The research gap

Three observations follow from this review.

First, existing studies differ substantially in the feature spaces they examine. Metaheuristic feature selection has been applied to naturally imbalanced transactional fraud data [16], [7], but generally on feature spaces that are relatively small or already transformed. Limited attention has been given to large and heterogeneous transactional datasets where high dimensionality, missing values, categorical variables, and correlated features occur simultaneously. IEEE-CIS provides such a setting.

Second, limitations remain in experimental design. Many studies evaluate feature-selection methods while keeping the wrapper classifier fixed: Agrawal et al. [15] show that two

classifiers account for roughly seven in ten wrapper evaluations, while Stephan et al. [18] compare nine selectors using a single k-nearest-neighbors classifier. Other studies focus primarily on comparisons among selected subsets rather than against the complete feature space. Although some studies evaluate feature selection across multiple classifiers [29], such comparisons remain limited, particularly for large and naturally imbalanced transactional datasets. This is important because the effectiveness of a selected subset may depend on the predictive model used, and dimensionality reduction alone does not establish an advantage over retaining all available predictors.

Third, statistical validation is applied inconsistently across feature selection studies. While some studies use tests such as Friedman or Wilcoxon, others rely only on observed performance differences, and correction for multiple comparisons also varies. This makes it difficult to determine whether reported improvements represent statistically meaningful differences between methods.

Based on these gaps, this thesis systematically compares established feature selection methods across multiple predictive models on the IEEE-CIS dataset, using imbalance-appropriate evaluation metrics and statistical testing.

Chapter 3. Methodology

This chapter describes the methodology used to evaluate feature selection across predictive models for e-commerce fraud detection using the IEEE-CIS dataset. Seven feature-selection methods and a no-selection baseline are evaluated across six classifiers, allowing the effect of reducing the feature space to be assessed rather than assumed. The chapter covers data preparation, feature selection, classification, class-imbalance handling, performance evaluation and statistical comparison.

3.1 Experimental Design

The experiment follows a fully crossed design with two factors: feature-selection condition and classifier. Eight feature conditions, consisting of seven selection methods and a no-selection baseline retaining all 433 predictors, are evaluated across six classifiers covering linear, neural-network and tree-based learning, resulting in 48 experiments on the same held-out test partition. The no-selection baseline provides the reference for assessing whether reducing the feature space improves or maintains predictive performance across different learning mechanisms. To support comparability, all conditions use the same chronological data partitions, a common budget of 120 predictors for the seven selection methods, the same cost-sensitive approach to class imbalance and fixed classifier configurations across feature conditions. Within each classifier, the feature set is therefore the element that changes across feature-selection conditions, allowing performance differences to be examined without changes in data partitioning, classifier configuration or evaluation procedure. Figure 3.1 summarizes the experimental pipeline.

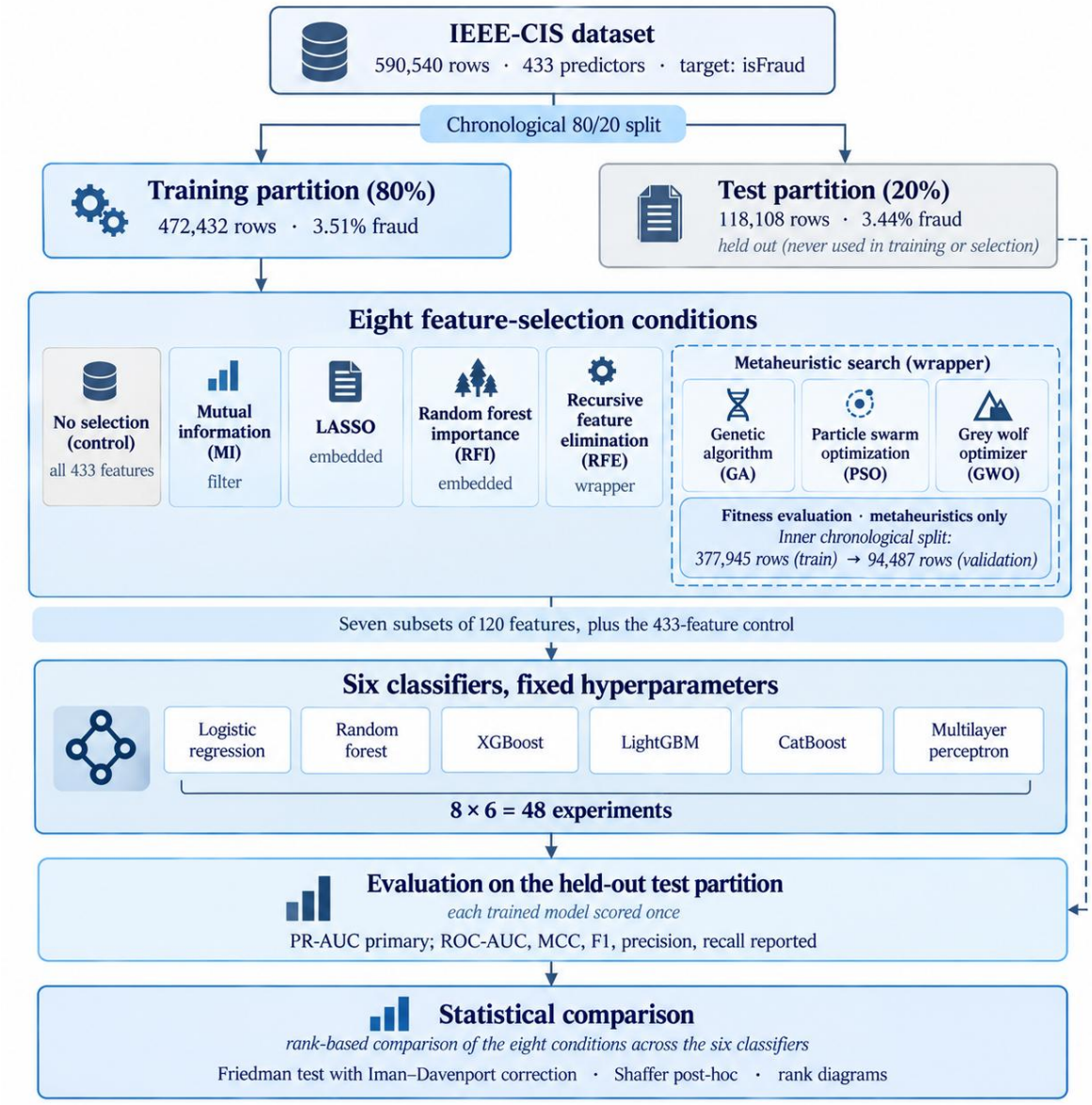


Figure 3.1. The experimental pipeline.

3.2 Data and Preprocessing

The preprocessing pipeline was designed to produce consistent model-ready data while preventing information from the later test period from entering model development. Following the chronological split, all preprocessing parameters were estimated from the training partition and applied unchanged to the test partition.

3.2.1 The Dataset and Feature Composition

The study uses the IEEE-CIS Fraud Detection dataset, released for a competition organized by the IEEE Computational Intelligence Society with data supplied by Vesta Corporation. The records describe card-not-present e-commerce transactions and are distributed as a transaction table containing payment attributes and the fraud label, and an identity table containing device, browser and network information. The dataset contains 590,540 labelled transactions, including 20,663 fraudulent transactions, a fraud rate of approximately 3.5%.

The identity table contains records for 144,233 transactions and was merged with the transaction table using a left join on TransactionID so that all transactions were retained. The merged data contain 434 columns.

Table 3.1. Predictor families in the merged dataset.

Family	Count	Content
TransactionDT	1	Seconds elapsed from an unrevealed reference date
TransactionAmt	1	Transaction value in US dollars
Card, address, email	12	Payment instrument, location and email domain attributes
C1–C14	14	Anonymized count variables
D1–D15	15	Anonymized time-delta variables
M1–M9	9	Boolean match indicators
V1–V339	339	Vesta-engineered variables
Identity block	41	Device, browser and network attributes

The V1–V339 variables account for approximately 78% of the predictor space and therefore represent a major source of dimensionality. An examination of their dependence structure showed a mean absolute correlation of 0.153 among computable variable pairs, with only 2.6% having an absolute correlation above 0.90. A principal component analysis provided a complementary view of this structure: 104 components were required to explain 95% of the variance in the V-variable block. Correlations among the V variables were generally weak, although a small proportion of variable pairs were strongly correlated. Together, these results indicate that variation in the V block is distributed across a relatively large number of dimensions.

3.2.2 Preprocessing Procedure

The preprocessing procedure consisted of the following steps. The Python code implementing each step is provided in Appendix D.

Step 1: Merge and remove the identifier.
The transaction and identity tables were merged using a relational left join on TransactionID. The TransactionID field was then removed because it serves as an identifier rather than a substantive predictor.

Step 2: Derive transaction hour.
The "TransactionDT" variable measures elapsed seconds from an undisclosed reference point and was retained as a candidate predictor and as the variable used to order transactions. TransactionHour was derived from this variable to represent the hour within the 24-hour daily cycle. This transformation was included because transaction and fraud activity vary by time of day:

$$\text{TransactionHour} = \left(\frac{TransactionDT}{3600}\right) \bmod 24$$

Step 3: Chronological split.
The transactions were first sorted by TransactionDT. An 80/20 chronological split then formed the training partition from the earliest 472,432 transactions, while the latest 118,108 were held out for final testing. This reflects the practical setting in which earlier transactions are used to predict later ones.

Step 4: Impute numerical variables.
To handle missing numerical values without introducing information from the test period, column medians were calculated from the training partition only. The same training medians were then applied unchanged to the test partition.

Step 5: Encode categorical variables.
Categorical variables were mapped to integer codes using only the levels observed in the training partition, preventing information from the later test period from influencing the encoding scheme. Missing categorical values were assigned a separate code of -1. Categories appearing in the test partition but not observed during training were assigned -2, allowing unseen categories to be represented without modifying the mappings learned from the training data.

Step 6: Construct the final feature matrices.
The `isFraud` target was separated from the predictor matrices, producing the training and test feature matrices (`X_train` and `X_test`) and their corresponding target vectors (`y_train` and `y_test`).

3.2.3 Resulting partitions

Table 3.2. Chronological partitions.

Partition	Transactions	Fraud rate	Role
Training (earlier 80%)	472,432	3.51%	Preprocessing, feature selection, model fitting
Test (later 20%)	118,108	3.44%	Single final evaluation

The fraud rate differs by only 0.07 percentage points between the two periods, so the class balance against which PR-AUC is measured is comparable in both. Class imbalance was handled during model fitting rather than by resampling.

3.3 Feature Selection Methods

This study evaluates seven feature-selection methods alongside a no-selection baseline retaining all 433 predictors. The methods were chosen to represent different selection mechanisms and definitions of predictor relevance. Mutual Information (MI) provides a model-independent univariate filter; LASSO and Random Forest Importance (RFI) perform embedded selection through linear regularization and tree-based importance, respectively; and Recursive Feature Elimination (RFE) applies iterative wrapper elimination. Among the metaheuristic approaches, Genetic Algorithm (GA) uses an evolutionary search mechanism, while Particle Swarm Optimization (PSO) and Grey Wolf Optimizer (GWO) use swarm-intelligence-based search strategies. These approaches are also represented in the feature-selection literature reviewed in Chapter 2, including applications to fraud detection and high-dimensional data. Evaluating them under a common feature budget allows the study to compare the subsets

produced by different selection mechanisms and to examine whether more complex search strategies provide predictive advantages over simpler filtering, regularization, importance-ranking and elimination approaches.

All feature-selection procedures were restricted to the training partition. MI, LASSO, RFI and RFE used all 472,432 training observations. For GA, PSO and GWO, the training partition was divided chronologically again: the earlier 377,945 observations were used to fit the fitness evaluator and the later 94,487 were used for validation. The held-out test partition was not used during feature selection.

3.3.1 Common subset size

All seven feature-selection methods were required to select the same number of predictors. A common budget of 120 predictors was used, which reduces the feature space by 72.3% while keeping a subset large enough to represent the different variable groups in the dataset. A preliminary check with MI-ranked subsets on a chronological validation split showed that performance increased steadily as predictors were added, with no clear plateau, so the data did not indicate a natural cut-off. The budget was therefore fixed in advance as a practical compromise between dimensionality reduction and retained information. Using the same budget for every method also ensures that differences between methods reflect the predictors selected. The value of 120 is a design choice for controlled comparison and is not claimed to be an optimal subset size.

3.3.2 Filter and embedded methods

Mutual information (MI). This method measures how much knowing the value of a predictor reduces uncertainty about whether a transaction is fraudulent. It equals zero when the predictor and the fraud label are independent and increases with the strength of their dependence. MI scores were calculated separately for each predictor on the training partition, and the 120 highest-scoring predictors were retained. It provides a model-independent ranking and can capture non-linear dependence, although it evaluates predictors individually.

LASSO. LASSO performs feature selection by applying an L1 penalty that shrinks less informative logistic-regression coefficients towards zero. Predictors were first standardized using parameters estimated from the training data because the penalty is sensitive to feature scale. The regularization strength was calibrated to the common feature budget: with the SAGA solver, (C=0.000383) resulted in 120 coefficients with absolute values greater than 0.0001, and the corresponding predictors formed the LASSO subset.

Random-forest importance (RFI). RFI evaluates predictor relevance through the contribution of each variable to impurity reduction across the trees of a random forest. A forest of 1,000 trees was fitted to the training partition using square-root feature sampling, a minimum of 10 observations for node splitting, a minimum of 5 observations per leaf, and balanced-subsample class weighting. Predictors were then ranked by their impurity-based importance, and the 120 highest-ranked variables were retained.

3.3.3 Recursive feature elimination

Recursive feature elimination (RFE) is a wrapper method that selects predictors through repeated model fitting and elimination. In this study, XGBoost served as the internal estimator. Starting with the complete predictor set, XGBoost was fitted and the 20 least important

predictors were removed at each iteration. The model was refitted on the reduced feature set, allowing predictor importance to be recalculated after each elimination step. This process continued until the common budget of 120 predictors was reached. Unlike a single importance ranking, the iterative procedure allows the relative importance of the remaining predictors to change as other variables are removed.

3.3.4 Metaheuristic search methods

GA, PSO and GWO were implemented as wrapper searches. Each candidate solution was a binary vector of length 433, where 1 indicated a selected predictor:

$$z = (z_1, \ldots, z_{433}) \in \{0,1\}^{433}, \quad \|z\|_0 = \Sigma_j z_j$$

The quality of each candidate subset was measured by fitting XGBoost on the inner training data and calculating ROC-AUC on a later chronological validation partition. The search objective:

$$z^* = max_z AUC_{ROC}(z), \quad 30 \le \|z\|_0 \le 120$$

To perform this evaluation without using the held-out test period, the original training partition was divided chronologically again. The earlier 377,945 observations were used to fit the XGBoost evaluator, while the subsequent 94,487 observations were used to calculate validation fitness.

The common XGBoost fitness evaluator used 200 trees, a maximum depth of 5, a learning rate of 0.1, row and column subsampling rates of 0.8, histogram-based tree construction and positive-class weighting based on the class ratio of the inner training partition. All three methods used the same feature pool, inner training and validation partitions, XGBoost evaluator and random seed. Each search used a population of 30 candidate solutions and ran for 50 generations or iterations, with candidate subsets allowed to contain between 30 and 120 predictors. The final subset retained from each method contained 120 predictors, consistent with the common feature budget. Under this design, the methods search for high-performing subsets within the same computational budget rather than evaluating all possible subsets.

Table 3.3. Settings of the metaheuristic searches.

Method	Search mechanism	Main settings
GA	Tournament selection, single-point crossover and bitwise mutation	Population 30; 50 generations; crossover 0.8; mutation 0.03; elitism 10%; 30-120 features
PSO	Particle updates guided by personal and global best positions; sigmoid transfer to binary	Population 30; 50 iterations; c1=c2=1.8; inertia 0.9→0.4; 30-120 features
GWO	Updates guided by alpha, beta and delta solutions; sigmoid transfer to binary	Population 30; 50 iterations; control parameter 2→0; 30-120 features

Genetic algorithm (GA). The initial population was generated randomly. In each generation, the best 10% of subsets were copied unchanged into the next generation. The remaining subsets were produced by choosing parents through binary tournament selection, in which the better of two randomly chosen subsets was selected, combining them through single-point crossover, and applying bit-flip mutation, which reverses the inclusion of each predictor with

a small probability. Subsets with more than 120 predictors were reduced by randomly removing selected predictors, and subsets with fewer than 30 received a fitness of zero.

Particle swarm optimization (PSO). A binary version of PSO was used. Each particle represented a subset and had a continuous velocity for every predictor. At each iteration, the velocity combined the particle's previous movement, weighted by a decreasing inertia, with movements towards its own best subset and the best subset found by the swarm. Velocities were converted into selection probabilities using a sigmoid function, and each predictor was then selected at random according to its probability. Subsets outside the 30–120 range were repaired by randomly adding or removing predictors.

Grey wolf optimizer (GWO). A binary version of GWO was used. In each iteration, the three best subsets found so far (alpha, beta and delta) guided the movement of the other wolves: each wolf's new position was the average of three positions computed relative to these leaders. A control parameter decreasing from 2 to 0 shifted the search gradually from exploring new regions to refining the best solutions. Positions were converted into selection probabilities using a sigmoid function, followed by random selection and the same repair step as in PSO. The leaders were retained between iterations, so the best subset was never lost.

3.3.5 Summary of conditions

Table 3.4. Summary of the eight feature-selection conditions.

Condition	Family	Selection criterion
No selection	Baseline	All 433 predictors retained
Mutual information	Filter	Top 120 by MI score
L1 logistic regression	Embedded	|coefficient| > 0.0001, C = 0.000383
Random-forest importance	Embedded	Top 120 by impurity importance, 1,000 trees
Recursive feature elimination	Wrapper	XGBoost ranking; 20 removed per step to 120
Genetic algorithm	Metaheuristic	ROC-AUC fitness; 30–120 features; 1,500 evaluations
Particle swarm optimization	Metaheuristic	ROC-AUC fitness; 30–120 features; 1,500 evaluations
Grey wolf optimization	Metaheuristic	ROC-AUC fitness; 30–120 features; 1,530 evaluations

3.4 Predictive Models

Each of the eight feature-selection conditions is evaluated using six classifiers, giving 48 experiments. The classifiers represent different learning mechanisms: a linear model, a bagging ensemble, three gradient-boosting methods and a neural network. Evaluating every subset with every classifier makes it possible to examine whether the effect of feature selection is consistent across models rather than specific to one learning algorithm.

3.4.1 The classifiers

Six classifiers representing different learning approaches were used to evaluate each feature-selection condition: Logistic Regression (LR), Random Forest (RF), XGBoost (XGB), LightGBM (LGBM), CatBoost (CAT) and a Multilayer Perceptron (MLP). The learning mechanism of each classifier and its role in the comparison are described below.

Logistic Regression (LR) provided the linear baseline and allowed the selected feature sets to be evaluated with a comparatively simple linear decision function. The model was fitted using the SAGA solver with balanced class weights, a maximum of 1,000 iterations and a convergence tolerance of 0.001. Because LR is sensitive to predictor scale, inputs were standardized using parameters estimated from the training data only.

Random Forest represented a tree-based bagging approach capable of modelling non-linear relationships and interactions between predictors. The model used 300 trees with unrestricted depth and square-root feature sampling. Class imbalance was handled using balanced_subsample, which recalculates class weights within each bootstrap sample. As tree-based models do not require standardized inputs, the original unscaled predictors were used.

XGBoost represented gradient boosting through sequentially constructed decision trees. The model used **500 trees**, with a learning rate of **0.05**, maximum depth of **6**, minimum child weight of **5**, row and column subsampling of **0.8**, an L2 penalty of **1**, and histogram-based tree construction. Class imbalance **was handled using the training-set class ratio**. XGBoost **was also used** as the estimator for RFE and as the fitness evaluator for the metaheuristic methods GA, PSO and GWO.

LightGBM provided a second gradient-boosting implementation using a leaf-wise tree-growth strategy. It was fitted for **500 boosting iterations** with a **learning rate of 0.05**, **maximum depth of 6**, 31 leaves and a minimum of 20 observations per leaf. Row and column subsampling rates were set to **0.8**, with an **L2 penalty of 1**. The class imbalance was handled using the training-set class ratio, and the original unscaled predictors were used.

CatBoost provided a third gradient-boosting implementation. Unlike XGBoost and LightGBM, it builds symmetric (oblivious) trees, in which the same splitting rule is applied at every node of a given depth. The model was trained with the log-loss objective for **500 iterations**, a **learning rate of 0.05** and a tree **depth of 6.** Class imbalance was handled through class weights of 1 for legitimate transactions and approximately 27.5 for fraudulent transactions, based on the training-set class ratio.

The multilayer perceptron (MLP) represented a non-linear neural-network model, providing a different learning structure from the linear and tree-based classifiers. The network contained three hidden layers with **128, 64 and 32** units and **ReLU** activation. It was trained using Adam with a learning rate of **0.001**, L2 regularization **(alpha = 0.0001)** and a batch size of 2,048. The training was limited to a maximum of **100 iterations**. Inputs were standardized using training-set parameters, while class imbalance was handled through observation level weights based on the training class ratio.

3.4.2 Model configurations

Table 3.5. Classifier configurations, held constant across all eight conditions.

Model	Configuration	Weighting	Scaled
Logistic regression	saga solver, maximum 1,000 iterations, tolerance 1e-3	balanced class weights	Yes
Random forest	300 trees, unrestricted depth, square-root feature sampling	balanced within bootstrap samples	No
XGBoost	500 trees, depth 6, learning rate 0.05, subsampling 0.8, minimum child weight 5, L2 penalty 1	positive-class weight	No
LightGBM	500 trees, depth 6, 31 leaves, learning rate 0.05, subsampling 0.8, minimum 20 samples per leaf	positive-class weight	No
CatBoost	500 iterations, depth 6, learning rate 0.05	explicit class weights	No
Multilayer perceptron	128–64–32 layers, ReLU, Adam, learning rate 0.001, L2 regularization 0.0001, batch size 2,048, max. 100 iterations.	per-sample weights	Yes

The standardization was applied to logistic regression and the MLP because their optimization is sensitive to differences in predictor scale:

$$Z_{ij} = \frac{X_{ij} - \mu_{j,train}}{\sigma_j}$$

where $\mu_{j,train}$ and σ_j are the mean and standard deviation of predictor j, estimated from the training data. The tree-based models used the original unscaled predictors.

A fixed random seed was used where supported. Threshold-dependent metrics for the 48 experiments used a common probability threshold of 0.50; a supplementary analysis with thresholds selected on validation data.

3.5 Class Imbalance

Fraud accounts for only about 3.5% of the observations. The study therefore used cost-sensitive learning instead of synthetic oversampling or majority class undersampling. The original observations and natural class distribution were retained, while errors on fraudulent transactions received greater weight during model fitting.

For XGBoost, LightGBM and CatBoost, the fraud class was assigned a weight based on the training-set class ratio:

$$w_+ = \frac{N_0}{N_1} \approx 27.5$$

where N_0 and N_1 are the numbers of non-fraudulent and fraudulent transactions in the training partition, respectively. Non-fraudulent transactions retained a weight of 1. Logistic Regression used balanced class weights, giving the same relative class-weight ratio of N_0/N_1. Random Forest used balanced weights recalculated within each bootstrap sample. For the MLP, the same weights were applied to individual observations.

All weights were calculated from the training data only, and the held-out test partition retained its original class distribution. This approach avoids introducing synthetic observations and preserves the chronological evaluation setting.

3.6 Evaluation Metrics

Each of the 48 fitted models was evaluated once on the same held-out test partition, after feature selection and model fitting had been completed using the training data only. The recorded outputs included accuracy, precision, recall, specificity, F1-score, ROC-AUC, PR-AUC, Matthews correlation coefficient (MCC), confusion-matrix counts, number of predictors and training time.

PR-AUC, reported in this study as Average Precision (AP), was the primary predictive measure because the positive class is rare and precision-recall evaluation focuses directly on fraud detection and false-positive burden. AP was computed using scikit-learn's `average_precision_score` as a weighted mean of precision values across thresholds, with the increase in recall used as the weight, rather than by trapezoidal interpolation of the precision-recall curve:

$$AP = \Sigma_n (R_n - R_{n-1}) P_n$$

where P_n and R_n are the precision and recall values at successive thresholds. The test set contains 4,064 fraudulent transactions out of 118,108, giving a baseline Average Precision of approximately 0.034 for random predictions [4]. ROC-AUC was also reported to provide a complementary assessment of model discrimination.

Precision, recall, specificity, F1-score and MCC were calculated using class predictions obtained at the common threshold of 0.50. The main definitions are:

$$\text{Precision} = TP / (TP + FP)$$

$$\text{Recall} = TP / (TP + FN)$$

$$\text{Specificity} = TN / (TN + FP)$$

$$F1 = 2 \times \text{Precision} \times \text{Recall} / (\text{Precision} + \text{Recall})$$

$$F2 = 5 \times \text{Precision} \times \text{Recall} / (4 \times \text{Precision} + \text{Recall})$$

$$\text{MCC} = (TP \times TN - FP \times FN) / \sqrt{[(TP + FP)(TP + FN)(TN + FP)(TN + FN)]}$$

PR-AUC was pre-specified as the primary measure for the statistical comparison. ROC-AUC was reported as a secondary measure, and was analyzed with the same statistical tests to examine whether the conclusions depended on the measure. MCC was reported as a complementary threshold-dependent measure. Training time and subset size were treated as descriptive measures of computational and dimensionality trade-offs.

3.6.1 Validation-Based Threshold Selection

A fixed probability threshold of 0.50 may produce different precision–recall trade-offs across classifiers, particularly when class weighting affects their predicted scores. A supplementary analysis therefore examined validation-based threshold selection for six representative configurations: RFE + RF, RFE + XGB, NO FS + RF, NO FS + XGB, NO FS + LightGBM, and MI + LR.

The original training partition was divided chronologically into an inner training set (377,945 observations) and a validation set (94,487 observations). Each model was fitted using the inner training data, and candidate thresholds from 0.01 to 0.99, in increments of 0.01, were evaluated on the validation set. Two thresholds were selected: one maximizing MCC, which accounts for all four components of the confusion matrix, and another maximizing F2, which gives greater weight to recall than precision. The selected thresholds were then applied to predictions from models refitted on the full training partition and evaluated on the unchanged test set.

3.7 Statistical Validation

The 48 experiments produce one PR-AUC value for each combination of eight feature-selection conditions and six classifiers. Because the analysis compares multiple conditions across a limited number of classifier blocks, a rank-based framework was used rather than relying on parametric assumptions. PR-AUC levels also differ across classifiers, so ranking the feature-selection conditions within each classifier allows the comparison to focus on their relative performance. The same procedure was repeated using ROC-AUC.

3.7.1 Friedman and Iman–Davenport tests

Within each block, treatments were ranked from best to worst according to PR-AUC, with rank 1 assigned to the highest value. The Friedman test was used to test whether the expected ranks were equal. For the main analysis, the six classifiers acted as blocks and the eight feature-selection conditions as treatments, allowing feature-selection conditions to be compared while accounting for differences across classifiers. A reversed supplementary analysis exchanged these roles and compared the six classifiers across the eight feature-selection conditions.

Because only six blocks were available in the main analysis and the Friedman statistic can be conservative in small-sample settings, the Iman–Davenport correction was also calculated. Kendall's W was reported as a descriptive measure of agreement among blocks. This provides information about how consistently the feature-selection conditions were ranked across classifiers, since a mean ranking is informative only if the classifiers order the conditions similarly.

3.7.2 Shaffer post-hoc comparisons

A rejected omnibus test indicates that the conditions are not all equal but does not identify which pairs differ. Comparing all eight feature-selection conditions gives 28 pairwise hypotheses, while comparing the six classifiers gives 15. Testing these without adjustment would inflate the family-wise error rate. Pairwise comparisons were therefore performed using Shaffer's static step-down procedure at $\alpha = 0.05$ [42]. The procedure controls the family-wise error rate while using the logical relationships among pairwise hypotheses, making it less conservative than applying the same Bonferroni threshold to every comparison.

Mean-rank diagrams were used to summarize the post-hoc results. Methods joined by a bar are interpreted as having insufficient evidence of a statistically significant difference after correction; the connection is not treated as proof of equivalence.

3.7.3 Feature-subset similarity

In addition to predictive performance, agreement between the selected feature subsets was examined using the Jaccard similarity index. For two subsets A and B, the index is defined as

$$J(A, B) = \frac{|A \cap B|}{|A \cup B|}$$

where the numerator is the number of predictors selected by both methods and the denominator is the number selected by at least one method. The index ranges from 0, indicating no shared predictors, to 1, indicating identical subsets.

Jaccard similarity was calculated for all 21 pairwise comparisons among the seven selected subsets. Because each subset contained 120 predictors, differences in the index reflect the number of predictors shared between methods. The measure was used descriptively to assess similarity between the selected feature sets.

3.8 Software and Reproducibility

All experiments were implemented in Python and run on the same Windows 11 CPU-only machine, so the recorded times are comparable across methods. The main environment included Python 3.13.7, scikit-learn 1.8.0 and XGBoost 3.3.0, together with NumPy, pandas, SciPy, LightGBM and CatBoost. The preprocessing code is reproduced in Appendix D.

A random seed of 42 was set in Python, NumPy and all model parameters that accept one. Each feature-selection script saved the names of the selected predictors, the settings used and the search time. The ranking-based methods also saved the full predictor ranking, and the metaheuristic methods saved the fitness history of the search. Each classifier script saved the evaluation metrics, the confusion-matrix counts, the number of predictors and the training time. The memory used by the training data was measured with pandas (`memory_usage(deep=True)`).

3.9 Experimental Validity

Experimental validity was supported by separating model development from final evaluation. The data were partitioned chronologically, and preprocessing parameters, including imputation values, categorical mappings and scaling parameters, were estimated from the training data only. Feature selection was also performed without access to the held-out test period, which was used only for final evaluation.

XGBoost was used within RFE and as the fitness evaluator for GA, PSO and GWO. To examine whether the resulting feature subsets were useful beyond the model involved in selection, every feature-selection condition was evaluated with six classifiers representing different learning approaches.

Several methodological choices should be considered when interpreting the results. GA, PSO and GWO optimized ROC-AUC during the feature-selection search, whereas PR-AUC was used as the primary measure in the final evaluation. Better internal search fitness therefore need not correspond to better held-out PR-AUC. Mutual information was computed with the

encoded predictors treated as continuous. For categorical predictors, however, the integer codes represent category labels rather than meaningful numerical distances. The MI results should therefore be interpreted with this implementation choice in mind. In addition, the common budget of 120 predictors was imposed to provide a consistent comparison between feature-selection methods and should not be interpreted as the optimal number of predictors for the dataset.

Finally, classifier configurations were held fixed across feature-selection conditions rather than tuned separately for each subset. This improves comparability between the experimental conditions but means that the study does not determine the best possible configuration for every feature-selection and classifier combination. The metaheuristic methods were also evaluated under fixed search budgets, so their final recorded solutions should not be interpreted as proof of global optimality.

Chapter 4. Results

This chapter presents the empirical results of the 48 experiments formed by crossing eight feature-selection conditions with six classifiers. The analysis first examines the complete performance grid and the behavior of the selected subsets, then compares classifier performance, validates the observed rankings statistically, and finally considers the trade-off between predictive performance, dimensionality and computational cost.

4.1 Overall Predictive Results

4.1.1 PR-AUC Across the 48 Experiments

The first analysis compares all feature-selection and classifier combinations using PR-AUC. This comparison identifies the strongest configurations and shows how the effect of feature selection varies across classifiers. The individual values behind every summary in this chapter are reported in Appendix B.

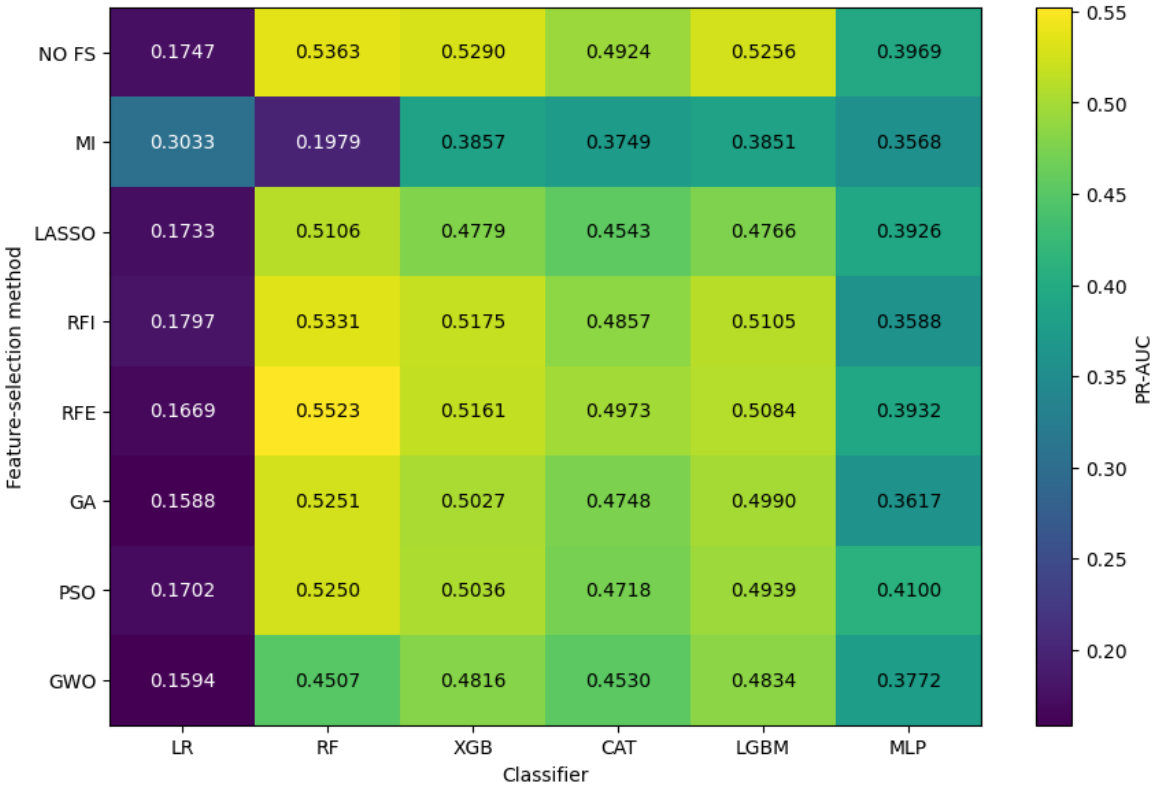


Figure 4.1. PR-AUC across the 48 feature-selection-classifier experiments

Figure 4.1 shows the PR-AUC obtained for every feature-selection and classifier combination. The strongest results are concentrated among the tree-based models. The single best result is RFE with Random Forest (PR-AUC = 0.5523), followed by the no-selection Random Forest model (0.5363) and RFI with Random Forest (0.5331). No-selection XGBoost (0.5290) and no-selection LightGBM (0.5256) are also among the strongest combinations.

The grid shows a clear interaction between feature selection and classifier choice. MI is the strongest example: it improves Logistic Regression from 0.1747 without selection to 0.3033, but reduces Random Forest from 0.5363 to 0.1979. More generally, four of the six classifiers

(Random Forest, CatBoost, MLP and Logistic Regression) reached their highest PR-AUC with a selected subset, while XGBoost and LightGBM performed best with the complete feature space. Apart from Logistic Regression, these gains were small, and the best method differed between classifiers. The effect of a selection method should therefore not be judged from a single classifier.

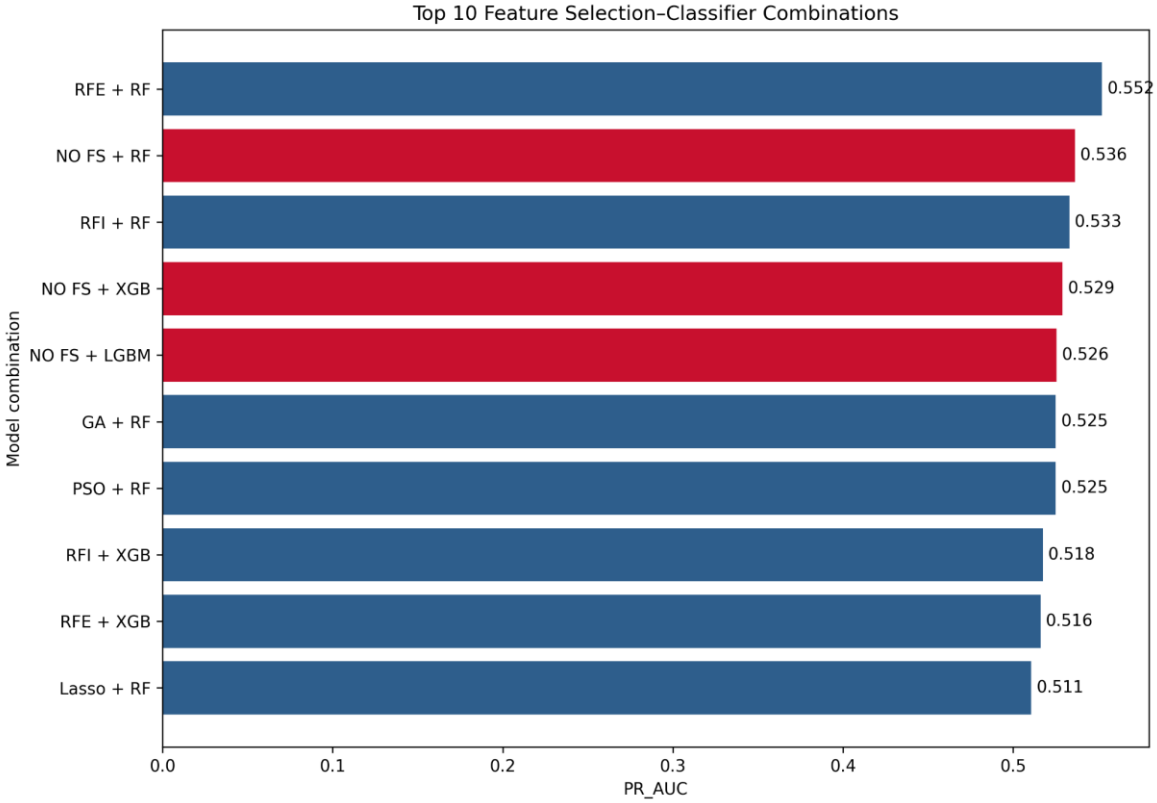


Figure 4.2. Ten highest-performing feature-selection-classifier combinations by PR-AUC

The ten strongest configurations have relatively similar PR-AUC values, making their distribution across classifiers and feature-selection methods particularly informative. Only three classifiers appear: Random Forest occupies six of the ten places, including the top three, XGBoost three and LightGBM one, while CatBoost, MLP and Logistic Regression do not reach the top ten under any feature condition. The feature conditions are more widely represented, with six of the eight appearing and the no-selection baseline occurring three times. Thus, high performance was not associated with a single feature-selection method, and reducing the feature space was not required to reach the strongest group. The concentration of the top ten among three tree-based classifiers, across several different feature representations, suggests that classifier choice is an important component of the observed performance pattern.

4.1.2 Discrimination under ROC-AUC and PR-AUC

ROC-AUC was also calculated for all 48 experiments. It measures how well the models separate fraudulent from legitimate transactions across thresholds and provides a complementary view of overall discrimination. Comparing it with PR-AUC shows whether the choice of metric changes the interpretation or ranking of the results.

Table 4.1. Mean ROC-AUC and mean PR-AUC by classifier, averaged over the eight selection conditions

Classifier	Mean ROC-AUC	ROC rank	Mean PR-AUC	PR rank
XGB	0.8886	2	0.4893	1
LGBM	0.8890	1	0.4853	2
RF	0.8725	4	0.4789	3
CAT	0.8854	3	0.4630	4
MLP	0.8261	5	0.3809	5
LR	0.8123	6	0.1858	6

Table 4.1 shows that the differences between classifiers are smaller in absolute terms under ROC-AUC than under PR-AUC. The difference between the best and worst classifier is 0.077 under ROC-AUC (LGBM against LR) but 0.304 under PR-AUC (XGB against LR). The two measures also order the classifiers differently: XGBoost and LightGBM exchange the first two positions, and Random Forest and CatBoost exchange the third and fourth. Logistic Regression illustrates this difference: its mean ROC-AUC of 0.812 indicates reasonable discrimination between fraudulent and legitimate transactions, while its mean PR-AUC of 0.186 reflects considerably weaker precision-recall performance.

Table 4.2. Mean ROC-AUC and mean PR-AUC by selection condition, averaged over the six classifiers

Condition	Mean ROC-AUC	ROC rank	Mean PR-AUC	PR rank
RFI	0.8815	1	0.4309	3
RFE	0.8808	2	0.4390	2
NO FS	0.8801	3	0.4425	1
PSO	0.8761	4	0.4291	4
GA	0.8751	5	0.4204	5
LASSO	0.8748	6	0.4142	6
GWO	0.8630	7	0.4009	7
MI	0.7673	8	0.3340	8

Table 4.2 shows that the two measures rank the feature-selection conditions similarly overall but differ among the leading methods. RFI ranks first under ROC-AUC and third under PR-AUC, while no selection moves from third to first; RFE is second under both. The ROC-AUC differences among these three conditions are very small (0.8801–0.8815).

At the individual configuration level, the best result differed between the two measures, but both came from RFE subsets. RFE with Random Forest achieved the highest PR-AUC (0.5523) and ranked fifth under ROC-AUC (0.9082), only 0.003 below the best value. RFE with XGBoost achieved the highest ROC-AUC (0.9112) and ranked ninth under PR-AUC (0.5161). The strongest configurations therefore remained among the leading models under both measures.

4.1.3 Threshold-Dependent Performance

PR-AUC and ROC-AUC evaluate predictive ranking across thresholds, while metrics calculated at a fixed probability threshold describe a specific classification rule. This section first examines the models at the default threshold of 0.50 and then presents a supplementary analysis using thresholds selected on validation data.

Table 4.3. Mean threshold-dependent metrics at probability threshold 0.50.

Classifier	MCC	Recall	Precision	F1	Accuracy
RF	0.4042	0.2508	0.7588	0.3453	0.9630
XGB	0.3551	0.6926	0.2204	0.3342	0.9048
LGBM	0.3415	0.7128	0.2018	0.3144	0.8931
CAT	0.3225	0.7247	0.1819	0.2907	0.8784
MLP	0.2673	0.6452	0.1534	0.2471	0.8624
LR	0.1990	0.7158	0.0977	0.1707	0.7539

At the default threshold of 0.50, Random Forest achieved the highest mean MCC (0.4042) and precision (0.7588), but low recall (0.2508). Other classifiers detected more fraud (recall of approximately 0.65–0.72), at the cost of lower precision. This illustrates how the same threshold produces different trade-offs across classifiers and helps explain why Random Forest led on MCC, which is calculated at a single threshold, while XGBoost led on PR-AUC, which evaluates performance across thresholds. Logistic Regression reached the highest individual recall (0.808 with RFI), but with a precision of only 0.081, reflecting the large number of transactions it flagged as fraud.

Table 4.3 reports mean accuracy, which ranged from 0.7539 for Logistic Regression to 0.9630 for Random Forest. However, accuracy should be interpreted carefully because 96.56% of test transactions are legitimate. A model that predicts every transaction as legitimate would achieve an accuracy of 0.9656, even though it would misclassify all fraudulent transactions. Accuracy was therefore reported as a complementary metric rather than used as the main basis for model comparison. Accuracy values for all 48 experiments are reported in Appendix Table B.5.

The corresponding differences between feature-selection conditions were smaller. Mean MCC spanned 0.107 across the eight conditions, compared with 0.205 across the six classifiers. The ordering of the conditions also differed from the PR-AUC ordering: PSO and GA achieved the highest mean MCC, while the no-selection condition ranked fifth. Performance at a single threshold therefore does not necessarily follow the same ordering as performance evaluated across thresholds.

To examine the effect of threshold choice, a supplementary analysis was conducted for six representative configurations. Thresholds maximizing MCC or F2 were selected on chronological validation data and then applied to the held-out test set. Table 4.4 compares these results with the default threshold of 0.50, while the validation results are reported in Appendix Table B.4.

Table 4.4. Test-set classification performance at the default and validation-selected thresholds.

Configuration	Threshold selection	Threshold	Precision	Recall	F1	MCC	Accuracy
RFE + RF	Default	0.50	0.879	0.240	0.377	0.451	0.973
	MCC-optimal	0.20	0.659	0.464	0.545	0.540	0.973
	F2-optimal	0.10	0.372	0.631	0.468	0.461	0.951
NO FS + RF	Default	0.50	0.882	0.230	0.365	0.442	0.972
	MCC-optimal	0.19	0.631	0.453	0.527	0.521	0.972
	F2-optimal	0.09	0.326	0.649	0.434	0.434	0.942
RFE + XGB	Default	0.50	0.237	0.728	0.357	0.382	0.910
	MCC-optimal	0.87	0.649	0.393	0.490	0.492	0.972
	F2-optimal	0.62	0.319	0.639	0.425	0.425	0.941
NO FS + XGB	Default	0.50	0.240	0.726	0.361	0.384	0.911
	MCC-optimal	0.87	0.665	0.397	0.497	0.501	0.972
	F2-optimal	0.65	0.355	0.625	0.453	0.446	0.948
NO FS + LGBM	Default	0.50	0.209	0.745	0.327	0.359	0.894
	MCC-optimal	0.88	0.681	0.403	0.507	0.512	0.973
	F2-optimal	0.69	0.358	0.610	0.451	0.443	0.949
MI + LR	Default	0.50	0.113	0.557	0.188	0.195	0.834
	MCC-optimal	0.91	0.556	0.251	0.346	0.359	0.967
	F2-optimal	0.64	0.193	0.454	0.271	0.258	0.916

Note: MCC-optimal and F2-optimal thresholds were selected on the validation set and applied once to the test set.

The selected thresholds differed considerably between classifiers. They were below 0.50 for Random Forest (0.09–0.20) and above 0.50 for the boosting models (0.62–0.88) and Logistic Regression (0.64–0.91). For Random Forest, lowering the threshold increased recall, while for the boosting models, raising it improved precision at the expense of recall.

Among the six configurations examined, RFE with Random Forest achieved the highest MCC under both threshold-selection rules. At the MCC-optimal threshold, its MCC increased from

0.451 to 0.540, while recall increased from 0.240 to 0.464. At the F2-optimal threshold, recall increased further to 0.631, although precision decreased to 0.372. With validation-selected MCC-optimal thresholds, RFE and the complete feature set produced similar MCC values for both Random Forest (0.540 and 0.521) and XGBoost (0.492 and 0.501). At the MCC-optimal thresholds, RFE with Random Forest also achieved higher recall than the boosting models (0.464 compared with 0.397–0.403) at similar precision, showing that the differences observed at the default threshold were largely related to threshold choice.

Accuracy, in contrast, was similar across all configurations at the MCC-optimal thresholds (0.967–0.973), even though their MCC ranged from 0.359 to 0.540, and it decreased at the F2-optimal thresholds as recall increased. This further shows that accuracy did not distinguish well between models with different fraud-detection ability.

4.2 Feature-Selection Outcomes

4.2.1 Overall Performance by Selection Condition

To compare the feature-selection conditions directly, PR-AUC is averaged across the six classifiers. This summary reduces the influence of any single classifier and provides a common view of each selector across the full experimental grid.

Table 4.5. Performance across the six classifiers by feature-selection condition.

Condition	Mean PR-AUC	SD	Mean rank
NO FS	0.4425	0.1411	1.83
RFE	0.4390	0.1436	2.83
RFI	0.4309	0.1383	3.17
PSO	0.4291	0.1328	4.17
GA	0.4204	0.1405	5.17
LASSO	0.4142	0.1244	5.67
GWO	0.4009	0.1244	6.33
MI	0.3340	0.0734	6.83

Note: Conditions were ranked from 1 (best) to 8 within each classifier, and the ranks were averaged across the six classifiers.

The no-selection condition achieved the highest mean PR-AUC (0.4425) and the best mean rank (1.83), followed by RFE with a mean PR-AUC of 0.4390 despite retaining only 120 predictors. The ordering by mean PR-AUC is identical to the ordering by mean rank, showing that the overall ordering is also reflected in the within-classifier rankings rather than arising only from differences in average classifier performance. Excluding MI, the seven remaining conditions fall within a relatively narrow range of 0.0416 in mean PR-AUC. In contrast, MI lies 0.0669 below GWO, the next lowest condition, a gap larger than the entire spread of the other seven conditions. MI also had the lowest standard deviation (0.0734), indicating a narrower spread across classifiers rather than stronger overall performance. Its mean rank of 6.83 reflects a notably classifier-dependent pattern: MI ranked first under Logistic Regression but last under the other five classifiers.

4.2.2 Metaheuristic Search Behavior

GA, PSO and GWO perform iterative population-based searches, so their recorded internal trajectories were examined separately. This analysis shows how quickly the best recorded solution improved and whether stronger internal fitness translated into stronger held-out PR-AUC.

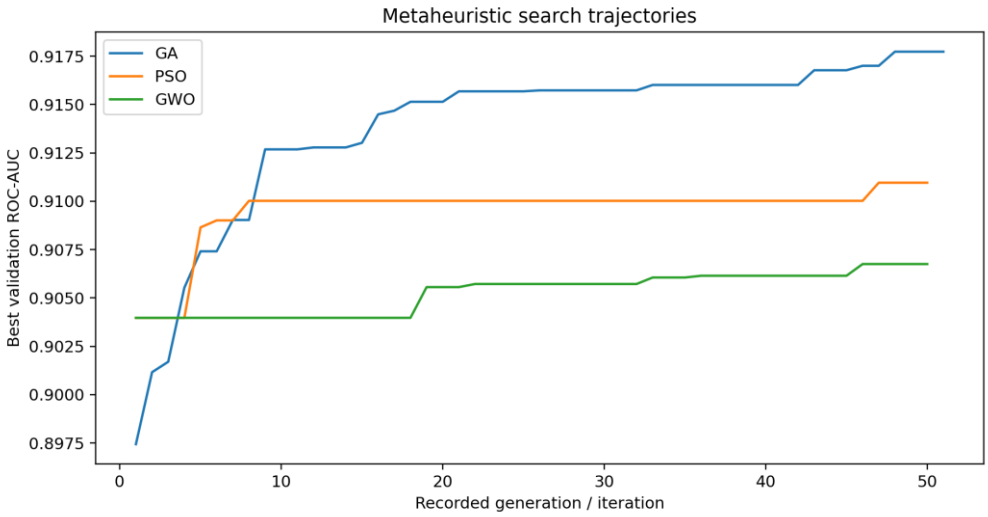


Figure 4.3. Recorded GA, PSO and GWO search trajectories using internal validation ROC-AUC.

Comparing the three search trajectories, GA improved gradually over the recorded generations and reached the highest final internal ROC-AUC (0.9177). PSO improved rapidly, obtaining about half of its total fitness improvement by iteration 5, whereas GWO required about 19 iterations. The recorded best solutions for PSO and GWO reached the upper subset bound of 120 features in the early iterations and remained at that size thereafter. Subsequent gains therefore mainly reflect changes in the composition of the best recorded subset rather than further changes in subset size.

Table 4.6. Internal search fitness and final held-out performance of the metaheuristic selectors.

Method	Final internal ROC-AUC	Held-out mean PR-AUC	FS mean rank
GA	0.9177	0.4204	5.17
PSO	0.9110	0.4291	4.17
GWO	0.9068	0.4009	6.33

Among the three metaheuristic methods, GA achieved the highest internal validation ROC-AUC, whereas PSO obtained the strongest held-out mean PR-AUC. Stronger internal fitness therefore did not necessarily translate into stronger held-out performance. Reaching the upper budget of 120 features does not imply that smaller candidate subsets were never explored during the search.

4.2.3 Overlap Between Selected Feature Subsets

The selected subsets were compared using Jaccard similarity to determine whether methods with the same 120-feature budget converged on similar predictors.

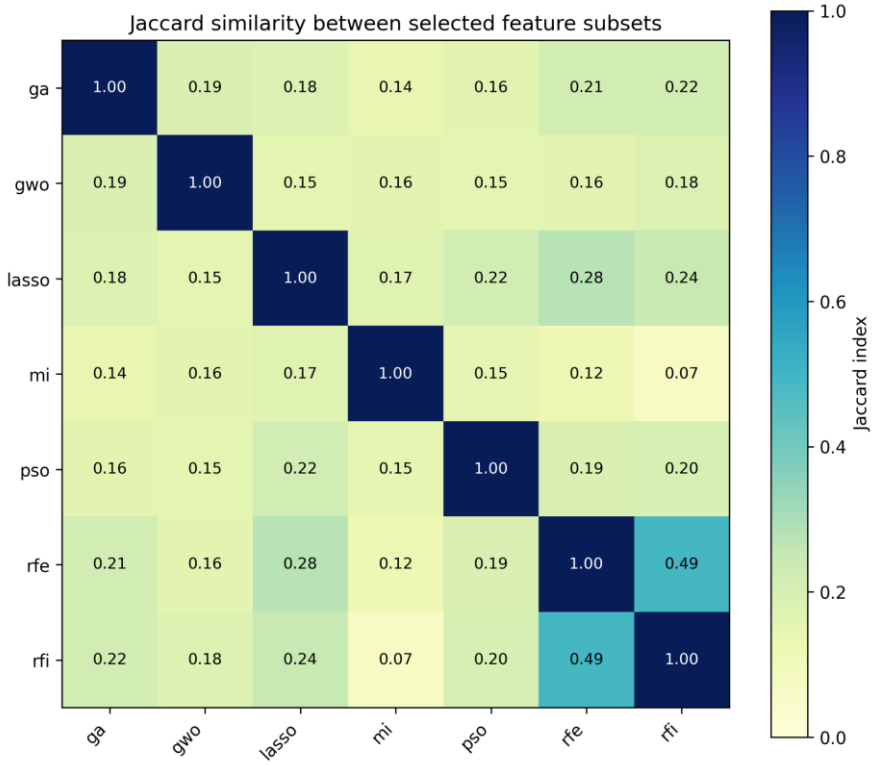


Figure 4.4. Jaccard similarity between the seven selected feature subsets.

Pairwise similarities were generally low, with an average of about 0.19. The largest overlap was between RFE and RFI (0.49), while the smallest was between MI and RFI (0.07). Thus, equal subset size did not imply similar feature composition, and the selectors often reached different representations of the same data.

4.2.4 Composition of the Selected Subsets

The selected predictors were grouped into the main IEEE-CIS feature families to compare the structure of the seven subsets.

Table 4.7. Feature-family counts in the selected subsets

Method	V	C	D	M	Identity	Card	Other
RFE	69	13	10	6	7	6	9
RFI	70	13	9	6	7	6	9
PSO	85	6	6	6	10	4	3
GA	86	6	3	5	11	5	4
LASSO	86	2	5	5	13	4	5
GWO	92	6	5	3	7	4	3
MI	97	0	1	0	18	2	2

Note. "Other" comprises the transaction, address, distance, email-domain, product-code and device fields.

V-features dominated all subsets, but their share varied. Proportional sampling would place about 94 V columns in a 120-feature subset. RFE and RFI retained only 69 and 70, favoring the count, date-delta and card fields, whereas GWO (92) and MI (97) retained close to or above the proportional share. MI also retained no count features or match flags, and excluded TransactionAmt, card1 and addr1, which were selected by most other methods.

4.3 Classifier Performance

Averaging across the eight feature-selection conditions helps determine which classifiers were strongest overall and which were more sensitive to the representation supplied to them.

Table 4.8. Classifier performance across the eight feature-selection conditions

Classifier	Mean PR-AUC	SD	Mean rank
XGB	0.4893	0.0453	1.88
RF	0.4789	0.1175	2.00
LGBM	0.4853	0.0434	2.62
CAT	0.4630	0.0391	3.75
MLP	0.3809	0.0202	4.88
LR	0.1858	0.0480	5.88

Note: Conditions were ranked from 1 (best) to 6 within each feature selection method, and the ranks were averaged across the 8 feature selection conditions.

XGBoost achieved the highest mean PR-AUC (0.4893) and the best mean rank (1.88), followed closely by RF and LightGBM. Although LightGBM had a slightly higher mean PR-AUC than RF (0.4853 vs 0.4789), RF achieved the better mean rank (2.00 vs 2.62), reflecting their different performance across individual selection conditions. RF also showed the greatest variability among the tree ensembles (SD = 0.1175), largely due to its sharp decline under MI, indicating greater sensitivity to the selected feature representation. The MLP and Logistic Regression achieved the lowest mean PR-AUC (0.3809 and 0.1858).

Under both PR-AUC and ROC-AUC, the tree-based ensembles, which build decision rules from many splits and can capture non-linear relationships and interactions between predictors, outperformed the MLP and Logistic Regression, the latter combining predictors linearly (Table 4.1).

4.4 Statistical Validation

The previous sections compared the performance and rankings of the feature-selection methods and classifiers. This section examines whether the observed ranking differences are statistically significant.

Two comparisons were made. First, the eight feature-selection conditions were compared across the six classifiers. Second, the six classifiers were compared across the eight feature-selection conditions. The Friedman and Iman–Davenport tests were used to check for overall

differences in rankings, while Kendall's W measured how consistently the methods were ranked across the different conditions. When significant differences were found, Shaffer's procedure was used for pairwise comparisons.

The analysis was first performed using PR-AUC, the primary evaluation metric, and then repeated using ROC-AUC as a secondary evaluation metric. Table 4.9 presents the overall test results, while the complete pairwise comparisons are provided in Appendix C.

Table 4.9. Friedman and Iman–Davenport tests on PR-AUC and ROC-AUC.

Comparison	Blocks	Treat.	χ^2	df	p	W	I-D F	p
Feature selection, PR-AUC	6	8	22.389	7	0.00218	0.533	5.708	0.000186
Classifiers, PR-AUC	8	6	30.286	5	1.3e-05	0.757	21.824	7.23e-10
Feature selection, ROC-AUC	6	8	31.222	7	5.66e-05	0.743	14.485	1.12e-08
Classifiers, ROC-AUC	8	6	31.429	5	7.71e-06	0.786	25.667	8.52e-11

For PR-AUC, the tests showed significant differences at $\alpha = 0.05$ among the feature-selection conditions (Friedman $\chi^2(7) = 22.389$, $p = 0.002$) and among the classifiers ($\chi^2(5) = 30.286$, $p < 0.001$). The Iman–Davenport tests confirmed these results, as shown in Table 4.9.

Kendall's W was 0.533 for feature selection and 0.757 for classifiers. This indicates that the classifiers were ranked more consistently across feature-selection conditions, while the relative performance of the feature-selection methods varied more between classifiers.

For ROC-AUC, significant differences were also found among feature-selection conditions ($\chi^2(7) = 31.222$, $p < 0.001$) and classifiers ($\chi^2(5) = 31.429$, $p < 0.001$). Kendall's W was 0.743 for feature selection and 0.786 for classifiers. Compared with PR-AUC, the higher W for feature selection indicates greater agreement in the rankings across classifiers under ROC-AUC.

The overall tests indicate that some ranking differences exist, but they do not identify which individual methods differ. These differences are examined through the Shaffer-adjusted pairwise comparisons.

4.4.1 Feature-Selection Conditions

For PR-AUC, only two of the 28 pairwise comparisons remained significant after Shaffer correction: NO FS versus MI (adjusted $p = 0.0114$) and NO FS versus GWO (adjusted $p = 0.0307$). Although NO FS achieved the best mean rank, its differences from RFE, RFI, and the other feature-selection methods were not statistically significant after correction.

For ROC-AUC, five comparisons remained significant. RFE and RFI both ranked significantly higher than MI (adjusted $p = 0.0010$) and GWO (adjusted $p = 0.0133$), while NO FS ranked significantly higher than MI (adjusted $p = 0.0133$).

Overall, the significant differences under both metrics all involved MI or GWO. No significant differences were found among NO FS, RFE, and RFI. However, this does not mean that these methods perform equally; it only means that the observed differences were not statistically significant in this analysis. With only six classifier blocks and correction across 28 comparisons, small rank differences are difficult to detect. The complete pairwise results are provided in Appendix C.

4.4.2 Classifiers

For PR-AUC, five of the 15 pairwise comparisons remained significant after Shaffer correction. Logistic Regression ranked significantly lower than XGBoost (adjusted p = 0.00029), Random Forest (0.00034), and LightGBM (0.0051). MLP also ranked significantly lower than XGBoost (0.0134) and Random Forest (0.0212).

For ROC-AUC, four comparisons remained significant. Logistic Regression ranked significantly lower than LightGBM (adjusted p = 0.00009) and XGBoost (0.00019), while MLP ranked significantly lower than LightGBM (0.0051) and XGBoost (0.0134).

Under both metrics, no significant differences were found among the tree-based classifiers. Although XGBoost, Random Forest, and LightGBM generally achieved the best rankings, none was significantly better than the other tree-based classifiers after correction, under either metric. The complete comparisons are reported in Appendix C.

4.5 Practical Performance-Parsimony-Cost Comparison

This section compares predictive performance, dimensionality reduction, and computational cost to assess the practical benefits of each feature-selection method.

Table 4.10. Performance retention and feature-selection cost relative to the no-selection control

Method	Mean PR-AUC	PR-AUC retained	ROC-AUC retained	Search (min)	Training (min)	Combined time (min)
NO FS	**0.4425**	100.0%	100.0%	–	24.8	24.8
RFE	**0.4390**	**99.2%**	100.1%	6.7	12.4	**19.1**
RFI	0.4309	97.4%	100.2%	6.8	12.6	19.4
PSO	0.4291	97.0%	99.5%	115.7	7.8	123.5
GA	0.4204	95.0%	99.4%	147.6	7.9	155.5
LASSO	0.4142	93.6%	99.4%	21.4	4.2	25.6
GWO	0.4009	90.6%	98.1%	111.8	7.6	119.4
MI	0.3340	75.5%	87.2%	7.0	5.9	12.9

Note: Training time represents the sum of fitting times for the six classifiers under each feature condition (Table B.1). Combined time includes feature-selection search and classifier training. PR-AUC retained and ROC-AUC retained give each condition's mean as a percentage of the no-selection mean.

RFE retained 99.2% of the no-selection mean PR-AUC while reducing the number of predictors by 72.3%. RFI achieved a similar reduction with slightly lower performance and nearly the same search time. In contrast, the metaheuristic methods required much longer searches, with combined times ranging from 119.4 to 155.5 minutes, without exceeding RFE in mean PR-AUC. MI had the shortest combined time (12.9 minutes) but retained only 75.5% of the no-selection mean PR-AUC.

For ROC-AUC, RFE and RFI slightly exceeded the no-selection mean, retaining 100.1% and 100.2%, respectively. The metaheuristic methods retained between 98.1% and 99.5%, while MI retained 87.2%.

The longer search times of the metaheuristic methods were mainly due to the repeated evaluation of candidate subsets. Each required approximately 1,500 XGBoost fits, compared with 17 for RFE. Reducing the feature space from 433 to 120 predictors also decreased the memory used by the training data, from approximately 1,505 MB to 407 MB for the RFE subset.

Across all six classifiers, RFE required 19.1 minutes of combined time, approximately 23% less than the 24.8 minutes needed without feature selection. Most of this saving came from Logistic Regression, whose training time decreased from 1,252.8 to 424.0 seconds. However, RFE did not reduce training time for every classifier: MLP training increased from 64.7 to 202.3 seconds.

For a single classifier, the cost of feature selection may outweigh the training-time savings. For example, RFE + RF required approximately 7.5 minutes in total, compared with 57.9 seconds for Random Forest without feature selection. These timings depend on the computing environment and whether the selected features are reused across multiple models.

Overall, RFE provided the best balance between dimensionality reduction, predictive performance, and measured computational cost under the experimental conditions of this study, although no selection achieved the highest mean PR-AUC.

4.6 Summary of Findings

RFE with Random Forest achieved the highest individual PR-AUC (0.5523), while RFE with XGBoost achieved the highest ROC-AUC (0.9112). Across classifiers, the no-selection condition achieved the highest mean PR-AUC, and RFE retained 99.2% of this performance while reducing the predictor set from 433 to 120 features. Under ROC-AUC, RFI and RFE achieved slightly higher mean scores than the no-selection condition. Statistical tests showed overall ranking differences under both metrics. However, no significant differences were found among RFE, RFI, and no selection after correction. The significant pairwise comparisons all involved MI or GWO.

Feature-selection effects varied across classifiers. MI, for example, improved Logistic Regression PR-AUC but reduced Random Forest substantially. The selected feature subsets also showed limited similarity. XGBoost achieved the highest mean PR-AUC across feature conditions, whereas Random Forest achieved the highest mean MCC at the 0.50 threshold. Using validation-selected thresholds changed the precision–recall trade-off. For RFE with Random Forest, MCC increased from 0.451 to 0.540, while recall increased from 0.240 to 0.464.

The computational analysis showed that RFE provided a favorable balance between predictive performance, dimensionality reduction, and measured computational cost. In contrast, the metaheuristic methods required substantially longer searches without exceeding RFE in mean held-out PR-AUC or ROC-AUC.

Across evaluation metrics, the same models appeared strong under ROC-AUC and accuracy but showed more modest results under PR-AUC, F1-score, and MCC. This highlights how the choice of evaluation metric affects the interpretation of model performance on imbalanced fraud data and supports the use of several complementary measures in this study.

Chapter 5. Discussion and Conclusion

This chapter discusses the main findings of the study, focusing on how feature selection affected different classifiers and the trade-off between predictive performance, dimensionality reduction, and computational cost. It also examines the role of evaluation metrics and compares the findings with previous research. Finally, it considers the practical implications, limitations and directions for future work before presenting the conclusions.

5.1 Interpretation of the Main Findings

5.1.1 Feature Reduction and Predictive Performance

RFE stood out among the selection methods. Combined with Random Forest, it achieved the highest individual PR-AUC in the study (0.5523, compared with 0.5363 without selection), although this difference was not tested separately for statistical significance, and combined with XGBoost it achieved the highest ROC-AUC (0.9112). It also reduced the predictor set by 72.3% while retaining 99.2% of the no-selection mean PR-AUC. One possible explanation is RFE's iterative procedure, which evaluates feature importance within the remaining subset at each step, unlike MI, which scores predictors individually and achieved the lowest mean PR-AUC.

Averaged across classifiers, however, feature selection did not improve PR-AUC over the complete feature space. The no-selection condition achieved the highest mean PR-AUC (0.4425) and the best mean rank (1.83), followed closely by RFE (0.4390) and RFI (0.4309). The Friedman and Iman–Davenport tests indicated an overall difference among the eight feature conditions. However, after Shaffer correction, the no-selection condition ranked significantly higher than only MI and GWO. The other five methods reduced the predictor set by 72.3%, with no statistically significant difference from the no-selection condition in this analysis. Under ROC-AUC, RFE and RFI ranked slightly above the no-selection condition and significantly higher than MI and GWO, while their difference from no selection was again not significant.

The structure of the IEEE-CIS feature space may help explain why the complete representation remained competitive. The V1–V339 block, representing approximately 78% of the predictors, had a mean absolute pairwise correlation of 0.153, while 104 principal components were needed to explain 95% of its variance. These findings suggest that variation was distributed across many variables rather than concentrated in a small number of components, which is consistent with the finding that reducing the feature space did not consistently improve predictive performance. Although both methods selected 120 predictors, RFE retained only 69 V variables and achieved the highest mean PR-AUC among the selection methods (0.4390), whereas MI retained 97 V variables and had the lowest (0.3340). Under the common feature budget, the differences in predictive performance therefore appear to reflect which predictors were retained rather than how many were removed.

Overall, feature selection should be assessed through both predictive performance and dimensionality reduction, rather than assuming that fewer predictors necessarily improve prediction large dataset. In this study, RFE provided a favorable balance between the two, while its combination with Random Forest achieved the highest individual PR-AUC. Whether these findings extend to other fraud datasets requires further evaluation.

5.1.2 Differences Between Feature-Selection Methods

If the effectiveness of feature selection depends on which predictors are retained, the selection criterion becomes particularly important. The methods differed in how they identified useful predictors. RFE and RFI, the two strongest of the seven selection methods on average, evaluated feature importance using tree-based models fitted within the selection procedure itself: XGBoost served as the internal estimator for RFE and Random Forest for RFI. In both methods, the importance of each predictor was therefore assessed while taking the other predictors into account. RFI produced a single importance ranking, whereas RFE repeatedly reassessed importance as predictors were removed. In contrast, MI evaluated each predictor individually based on its association with fraud, without considering its contribution alongside other predictors. These different approaches can lead to different decisions about the same variable.

A clear example appeared among the nine identity variables with more than 95% missing values. MI retained all nine, whereas RFE and RFI retained none. Such variables may still carry information about fraud, including through their pattern of missingness, which MI can capture when evaluating each variable individually. Model-based methods, in contrast, may assign them lower importance when other predictors already provide similar information. The same variables can therefore be treated very differently depending on the selection criterion.

The selected subsets also showed considerable differences, with a mean Jaccard similarity of only 0.19 and the highest agreement between RFE and RFI at 0.49. The more extensive searches performed by GA, PSO and GWO did not produce stronger average held-out performance than RFE under either PR-AUC or ROC-AUC, and GWO ranked significantly lower than both RFE and RFI under ROC-AUC. Their internal ordering also did not carry over to the final evaluation: GA achieved the highest internal fitness, whereas PSO achieved the best held-out mean PR-AUC. Stronger performance on the internal search objective therefore did not necessarily translate into stronger held-out performance.

5.1.3 Interaction with Classifier Choice

The usefulness of a selected subset also depended strongly on the classifier. MI increased Logistic Regression PR-AUC from 0.1747 without selection to 0.3033 but reduced Random Forest PR-AUC from 0.5363 to 0.1979. The same selected subset therefore improved one classifier while substantially reducing the performance of another. This shows why feature-selection methods should not be judged independently of the models that will use their selected variables.

Across all conditions, XGBoost achieved the best mean PR-AUC and mean rank, whereas Random Forest with RFE produced the highest individual PR-AUC. Under ROC-AUC, LightGBM achieved the highest mean value, closely followed by XGBoost. The post-hoc analysis did not find significant differences among the tree ensembles under either metric, so no single ensemble should be considered universally better. Instead, XGBoost was more consistently strong across feature conditions, while Random Forest produced the strongest individual configuration.

The stronger average performance of the tree-based ensembles in this experiment is consistent with their ability to represent nonlinear relationships and interactions among predictors, unlike the linear additive structure of Logistic Regression. The multilayer perceptron can also represent nonlinear relationships, but it performed less well than the tree-based ensembles.

This is consistent with the findings of Grinsztajn, Oyallon and Varoquaux [40], who reported that tree-based models generally outperform neural networks on tabular data.

5.1.4 Performance, Parsimony and Computational Cost

Feature selection can still be useful even when it does not increase average predictive performance. RFE removed 72.3% of the predictors while retaining 99.2% of the no-selection mean PR-AUC and 100.1% of its mean ROC-AUC. Its search took approximately 6.7 minutes, similar to RFI (6.8 minutes), while GA, PSO and GWO required approximately 112–148 minutes without exceeding RFE in average held-out PR-AUC. All reported times refer to the CPU used in this study.

This does not mean that the metaheuristic methods selected weak subsets. PSO and GA retained 97.0% and 95.0% of the no-selection mean PR-AUC and over 99% of its mean ROC-AUC, and they achieved the highest mean MCC at the 0.50 threshold, although these differences were small and were not tested statistically. Their main disadvantage was therefore computational cost rather than predictive performance, as RFE reached a comparable result with a much shorter search. GWO performed less well than the other metaheuristic methods, particularly under PR-AUC.

The reduced feature set also shortened model fitting. Across the six classifiers, fitting took 12.4 minutes with RFE compared with 24.8 minutes using all 433 predictors, giving a combined time of 19.1 minutes including the RFE search. This saving came mainly from Logistic Regression. For Random Forest alone, using all predictors was faster (57.9 seconds) than RFE + RF (approximately 7.5 minutes, including the 6.7-minute search). However, RFE + RF achieved a higher PR-AUC (0.5523 versus 0.5363) with 72.3% fewer predictors.

RFE therefore provided a useful compromise, reducing the feature space substantially with almost no loss in average predictive performance. Its practical value lay in reducing the number of predictors and requiring a relatively short search compared with the metaheuristic methods, rather than in improving average predictive performance. Reducing the number of predictors from 433 to 120 also lowered the memory used by the training data by about 73%, from approximately 1,505 MB to 407 MB for the RFE subset.

5.1.5 Interpreting the Performance Level

The choice of evaluation metric strongly influenced how model performance appeared. ROC-AUC and accuracy are widely reported in fraud-detection studies [16], [22], and both produced high values in this study: the best ROC-AUC was 0.911 and accuracy reached 0.973. However, these measures can give an overly optimistic impression of performance when fraud is rare. Although ROC-AUC evaluates discrimination across thresholds, its false-positive rate is calculated relative to the large number of legitimate transactions. With more than 114,000 legitimate transactions in the test set, even thousands of false alarms can correspond to a relatively small false-positive rate. Accuracy is more directly affected by class imbalance: a model predicting every transaction as legitimate would achieve 96.56% accuracy without detecting any fraud. A similar gap between ROC-AUC and PR-AUC was reported by Alamri and Ykhlef [5].

Measures that focus more directly on fraud detection gave a more moderate picture. The best PR-AUC was 0.552, while MCC and F1 reached 0.540 and 0.545, respectively, in the supplementary threshold analysis. These values indicate meaningful but imperfect predictive performance. For RFE with Random Forest, precision was approximately 0.90 at 20% recall

and 0.76 at 40% recall, but decreased to about 0.40 at 60% recall and 0.17 at 80% recall. This shows that the most suspicious transactions could be identified with relatively few false alarms, while detecting a larger share of fraud required accepting substantially more false positives. The low fraud prevalence of approximately 3.4% makes this trade-off particularly important. These results were also obtained under a chronological evaluation design, with preprocessing and feature selection restricted to the training data and classifier hyperparameters kept fixed.

The validation-based threshold analysis provided another important finding. For RFE with Random Forest, selecting the threshold to maximize MCC increased the test MCC from 0.451 to 0.540 and recall from 0.240 to 0.464, while precision decreased from 0.879 to 0.659. Using the F2-optimal threshold increased recall further to 0.631, but reduced precision to 0.372. Both thresholds were selected using validation data and then evaluated on the held-out test set. These results illustrate the trade-off between detecting more fraudulent transactions and generating more false alarms. They also show that the default threshold of 0.50 is not necessarily the most appropriate choice for fraud detection.

Overall, the evaluation measures highlighted different aspects of model performance. ROC-AUC indicated strong ranking discrimination, while accuracy was less informative because of class imbalance. PR-AUC, F1-score, and MCC provided a clearer picture of the difficulty of detecting fraud while limiting false alarms. Relying on ROC-AUC or accuracy alone could therefore have overstated the practical usefulness of the models, supporting the choice of PR-AUC as the primary evaluation metric alongside complementary measures. The appropriate decision threshold ultimately depends on the relative costs of missed fraud and false alarms, which should be considered when applying these models in practice.

5.2 Practical Implications for Feature Selection

Whether feature selection is worth applying depends on the structure of the data and the purpose of reducing the feature space. Before applying it, it is useful to examine redundancy, missing-value patterns, variability among predictors, and whether the variables are raw measurements or already engineered. In the IEEE-CIS data, strong pairwise correlations were relatively uncommon among the V1–V339 variables, which may partly explain why the complete feature space remained competitive. When the full feature set is manageable and predictive performance is the main concern, selection may offer little advantage.

Preprocessing is also important because it can influence which predictors a selection method identifies as useful. In this study, the data were divided chronologically, and preprocessing parameters were estimated from the training data before being applied to the later test data. This reduced the risk of information from the future test period influencing model preparation. The different treatment of highly missing variables by MI and the tree-based selectors also shows why feature selection should be considered together with preprocessing.

When dimensionality reduction is needed, a more complex selection method is not necessarily preferable. RFE provided the strongest practical balance among the evaluated selectors, retaining 99.2% of the no-selection mean PR-AUC and 100.1% of its mean ROC-AUC while reducing the number of predictors by 72.3%. The metaheuristic methods produced competitive subsets but required substantially more computation without improving average test performance. These results suggest that computational cost should be considered alongside predictive performance when choosing a selection method.

The selected feature subset should also be considered together with the classifier, since the same selection method can perform differently across models. Among the classifiers, tree-based ensembles provided the strongest results in this study. XGBoost achieved the highest mean PR-AUC and best mean rank across feature conditions, while LightGBM achieved the highest mean ROC-AUC. However, Random Forest produced the three highest individual PR-AUC values: RFE + RF (0.5523), NO FS + RF (0.5363), and RFI + RF (0.5331). This suggests that Random Forest is also worth considering with selected feature subsets, particularly RFE, when dimensionality reduction is desirable. Since the statistical analysis did not establish significant differences among the leading tree-based ensembles, the final choice should depend on validation performance and practical modelling requirements.

The threshold analysis also showed that performance at the default threshold does not necessarily reflect the best operating point for each classifier. At 0.50, NO FS + XGBoost achieved higher recall (0.73) but lower precision (0.24), while RFE + Random Forest achieved higher precision (0.88) but lower recall (0.24). After selecting MCC-optimal thresholds on validation data, their precision became similar (0.66–0.67), while RFE + Random Forest achieved slightly higher recall (0.46 compared with 0.40). These changes did not affect PR-AUC, but they changed the balance between missed fraud and false alarms.

5.3 Relation to Previous Research

The findings are broadly consistent with previous research on feature selection for fraud detection. Priscilla and Prabha [22], who also used the IEEE-CIS dataset, combined mutual information with XGBoost-based recursive feature elimination and reported an improvement in XGBoost ROC-AUC from 0.798 to 0.855. In the present study, XGBoost's ROC-AUC increased slightly from 0.9105 with all predictors to 0.9112 with the RFE subset, while RFE removed 72.3% of the predictors. Since the studies differ in their selection procedures and evaluation designs, the results are not directly comparable. However, both suggest that RFE can reduce dimensionality while maintaining predictive performance. The smaller improvement observed here may partly reflect the stronger complete-feature baseline.

The competitiveness of the complete feature space is also consistent with other studies. Wang et al. [24] found that larger feature subsets generally outperformed smaller ones on credit card fraud data, while El Barakaz Fatima et al. [28] reported that sparse selection without resampling did not exceed the no-selection baseline. In contrast, Ileberi et al. [16] and Mienye and Sun [7] reported benefits from genetic-algorithm-based selection. Both studies used the ULB credit card dataset, whose 28 transformed variables differ substantially from the larger and more heterogeneous IEEE-CIS feature space. These differences may help explain why the metaheuristic methods did not provide an advantage in the present study.

Overall, these comparisons suggest that the benefit of feature selection depends on the dataset, the classifier, and the baseline used. In this study, no selection method exceeded the complete feature space in mean PR-AUC, although RFE achieved substantial dimensionality reduction with limited predictive loss.

5.4 Scope and Boundaries of the Findings

The results come from the IEEE-CIS fraud detection dataset, which is large, and most of its predictors were engineered and anonymized by the data provider rather than taken directly from raw transaction fields. The exploratory analysis also showed that strong pairwise correlations were relatively uncommon in the V block: the mean absolute correlation was

0.153, only 2.6% of pairs exceeded 0.90, and 104 principal components were needed to explain 95% of the variance. These findings suggest that variability is distributed across many predictors, which could partly explain why reducing the feature space from 433 to 120 predictors did not improve average predictive performance. However, this analysis does not establish how much predictive information each variable contributes. On datasets with more redundant or uninformative variables, feature selection may produce different results, so the ranking of the methods should not be assumed to transfer without further testing.

Several decisions were fixed to maintain comparability across the 48 experimental configurations. First, all seven selectors used the same budget of 120 predictors, calibrated using MI-ranked subsets. Since MI later produced the weakest mean PR-AUC, a different calibration approach might have suggested another budget. The results therefore compare the methods at one level of reduction rather than at their individually optimal subset sizes. Second, classifier settings were held constant across feature conditions rather than tuned separately for each subset, and no additional feature engineering was performed beyond the derived TransactionHour variable. Consequently, the results do not establish the best performance achievable for each configuration, and the moderate values observed under PR-AUC, F1 and MCC should be interpreted in the context of this controlled experimental design.

Missing numerical values were replaced with training medians, and categorical variables were encoded using a common procedure. This standard approach gave all classifiers the same inputs and allowed a fair comparison. As a robustness check, the boosting models and Random Forest were also trained without imputation or manual encoding, using their native handling of missing values and categorical variables, both on the complete feature space and on the RFE subset. PR-AUC changed by at most about 0.01 and ROC-AUC by at most about 0.006, and the comparison between RFE and no selection was unchanged (Appendix Table B.6). The preprocessing approach therefore did not appear to affect the main conclusions, although the other selection methods were not re-run under native handling.

The threshold analysis was supplementary and exploratory, covering six configurations selected partly based on the main experimental results rather than all 48. Thresholds were selected on a single chronological validation split using two criteria, MCC and F2. The selected thresholds were then applied to the held-out test set.

The study used one chronological train–test split, which preserved temporal ordering and reduced the risk of temporal leakage but measured performance over a single later period. Each of the 48 configurations was also run once with a fixed seed, so variation across repeated runs was not estimated. The statistical comparison of feature-selection methods used six classifiers as blocks, and only two pairwise differences remained significant under PR-AUC after Shaffer's correction, compared with five under ROC-AUC. Because the ROC-AUC tests were based on the same models, they provide a consistency check rather than independent evidence. In addition, since all comparisons were based on a single dataset, the statistical findings describe ranking differences within the experimental design rather than generalization across independent datasets. With this limited number of blocks, smaller differences may have been difficult to detect. Therefore, the absence of a significant difference between no selection and most of the stronger selectors should not be interpreted as evidence that their performance is equivalent. Evaluation across additional temporal periods and repeated runs would provide stronger evidence about the stability of these findings.

5.5 Recommendations and Directions for Future Work

The findings suggest that the need for feature selection depends not only on the number of predictors but also on the redundancy, missingness, and information content of the feature space. Before reducing a high-dimensional dataset, its structure should therefore be examined, for example by checking redundancy, missingness, and how performance changes as more predictors are added. When reduction is needed, a method such as RFE is worth considering. In this study, RFE maintained strong predictive performance at a substantially lower computational cost than the metaheuristic searches. A more expensive search should have a clear expected benefit, rather than being chosen simply because it explores more subsets. Fraud-detection models should also be evaluated using PR-AUC as a primary measure, alongside complementary metrics such as MCC, precision, and recall, rather than relying on ROC-AUC or accuracy alone.

Several directions could improve predictive performance beyond the controlled design used here. First, the hyperparameters of each classifier could be tuned for each selected subset on a chronological validation set, to examine whether the ranking of the selection methods remains the same after tuning. Second, new features could be engineered with domain experts, such as transaction frequency or typical spending patterns for the same card or customer over time, using only information available at the time of each transaction to avoid leakage. Third, decision thresholds could be chosen based on the actual costs of missed fraud and false alarms rather than general criteria such as MCC or F2.

The robustness of the results also deserves attention. The selection methods could be compared at several subset sizes rather than a single budget of 120 predictors. Repeating each configuration with several random seeds would show whether the selected subsets and the small differences between conditions are stable. Finally, evaluating feature selection over successive time windows, and potentially on other fraud datasets, would help assess whether the findings remain consistent. Including the cost of repeating feature selection as transaction patterns change would also give a clearer view of its practical value in deployed fraud-detection systems.

5.6 Conclusion

This study examined whether feature selection improves fraud-detection performance or mainly provides a way to reduce the number of predictors. Seven feature-selection methods and a no-selection baseline were evaluated across six classifiers using the IEEE-CIS dataset, resulting in 48 experimental configurations. The comparison considered predictive performance, dimensionality reduction, computational cost, and statistical differences. The main finding was that RFE achieved the strongest individual results under both evaluation metrics: the highest PR-AUC (0.5523) with Random Forest and the highest ROC-AUC (0.9112) with XGBoost, while using 72.3% fewer predictors. However, across classifiers, the no-selection condition achieved the highest mean PR-AUC (0.4425), followed closely by RFE (0.4390).

RFE, using XGBoost as its internal estimator, reduced the feature space from 433 to 120 predictors while retaining approximately 99.2% of the no-selection mean PR-AUC. No statistically significant difference was found between RFE and no selection under either PR-AUC or ROC-AUC, although this does not establish that their performance was equivalent. RFE also reduced the memory required to store the training feature matrix by approximately 73%, with a search time of 6.7 minutes. Its search required only 17 internal model fits, compared

with approximately 1,500 for each metaheuristic method. Although the metaheuristic methods produced competitive subsets, their searches took around 17–22 times longer without achieving higher average held-out performance.

The effect of feature selection also depended strongly on the classifier. Four of the six classifiers achieved their highest PR-AUC with selected features, although the best selection method differed between them. XGBoost achieved the highest mean PR-AUC across feature conditions, while Random Forest produced the three highest individual PR-AUC values (with RFE, no selection and RFI). The effect of MI was particularly clear: it improved Logistic Regression PR-AUC from 0.1747 to 0.3033 but reduced Random Forest PR-AUC from 0.5363 to 0.1979. This shows that a feature subset that works well for one classifier may be less suitable for another.

The evaluation results also showed why the choice of metric matters in fraud detection. With only 3.44% fraudulent transactions in the test set, predicting every transaction as legitimate would already achieve 96.56% accuracy. The models reached higher accuracy values of up to 0.973 and ROC-AUC of 0.9112, but these measures alone did not fully describe the difficulty of detecting fraud while limiting false alarms. PR-AUC was therefore appropriate as the primary metric because it summarizes the precision–recall trade-off across thresholds, focusing on the minority class. The highest PR-AUC of 0.5523 was approximately 16 times the random ranking baseline of 0.0344, indicating substantial predictive performance despite the difficulty of the task. MCC, F1, precision and recall provided complementary information about classification performance at specific thresholds. For RFE with Random Forest, selecting an MCC-optimal threshold on validation data increased test recall from 0.240 to 0.464 and MCC from 0.451 to 0.540, while precision decreased from 0.879 to 0.659. The share of fraud detected (recall) therefore depended largely on the chosen threshold. This illustrates why both threshold-independent ranking metrics and threshold-dependent classification metrics are needed to evaluate fraud-detection models.

Overall, the findings show that the value of feature selection extends beyond improving predictive scores. Keeping the complete feature space can remain a reasonable choice in large datasets such as IEEE-CIS, where it achieved the highest mean PR-AUC, but retaining hundreds of anonymized predictors makes the models harder to interpret and more demanding to store, process and maintain. A well-chosen method such as RFE can therefore provide a better balance between predictive performance, dimensionality reduction and computational cost. In high-dimensional datasets with greater redundancy or noise, this balance may favor feature selection even more strongly.

References

[1] Dastidar, K. G., Caelen, O. and Granitzer, M., "Machine learning methods for credit card fraud detection: a survey," IEEE Access, 2024.

[2] Diaconu, A. and Pele, D. T., "Fraud detection on the IEEE-CIS dataset: gradient boosting against graph neural networks," Proceedings of the International Conference on Business Excellence, 2026.

[3] Davis, J. and Goadrich, M., "The relationship between precision-recall and ROC curves," Proceedings of the 23rd International Conference on Machine Learning, 2006.

[4] Saito, T. and Rehmsmeier, M., "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," PLoS ONE, 2015.

[5] Alamri, M. and Ykhlef, M., "A novel hybrid machine learning model for credit card fraud detection using anomaly detection and optimised classification," PeerJ Computer Science, vol. 12, e3748, 2026, doi: 10.7717/peerj-cs.3748.

[6] Boughorbel, S., Jarray, F. and El-Anbari, M., "Optimal classifier for imbalanced data using Matthews Correlation Coefficient metric," PLoS ONE, 2017.

[7] Mienye, I. D. and Sun, Y., "A machine learning method with hybrid feature selection for improved credit card fraud detection," Applied Sciences, 2023.

[8] Zhao, Z. and Bai, T., "Financial fraud detection and prediction in listed companies using SMOTE and machine learning algorithms," Entropy, 2022.

[9] Saputra, A. and Suharjito, "Fraud detection using machine learning in e-commerce," International Journal of Advanced Computer Science and Applications, 2019.

[10] Esenogho, E., Mienye, I. D., Swart, T. G., Aruleba, K. and Obaido, G., "A neural network ensemble with feature engineering for improved credit card fraud detection," IEEE Access, 2022.

[11] Tran, B., Zhang, M. and Xue, B., "A PSO based hybrid feature selection algorithm for high-dimensional classification," IEEE Congress on Evolutionary Computation, 2016.

[12] Dal Pozzolo, A., Caelen, O., Johnson, R. A. and Bontempi, G., "Calibrating probability with undersampling for unbalanced classification," IEEE Symposium Series on Computational Intelligence, 2015.

[13] Al-Tashi, Q., Md Rais, H., Abdulkadir, S. J., Mirjalili, S. and Alhussian, H., "A review of grey wolf optimizer-based feature selection methods for classification," Evolutionary Machine Learning Techniques, Springer, 2020.

[14] Ng, A. Y., "Feature selection, L1 vs. L2 regularization, and rotational invariance," Proceedings of the 21st International Conference on Machine Learning, 2004.

[15] Agrawal, P., Abutarboush, H. F., Ganesh, T. and Mohamed, A. W., "Metaheuristic algorithms on feature selection: a survey of one decade of research (2009-2019)," IEEE Access, 2021.

[16] Ileberi, E., Sun, Y. and Wang, Z., "A machine learning based credit card fraud detection using the GA algorithm for feature selection," Journal of Big Data, 2022.

[17] Emary, E., Zawbaa, H. M. and Hassanien, A. E., "Binary grey wolf optimization approaches for feature selection," Neurocomputing, 2016.

[18] Stephan, P., Punitha, S., Lin, J. C.-W. and Agarwal, B., "A comprehensive study of grey wolf optimizer variants for optimizing feature selection in high-dimensional data," Applied Artificial Intelligence, 2026.

[19] Siam, A. M., Bhowmik, P. and Uddin, M. P., "Hybrid feature selection framework for enhanced credit card fraud detection using machine learning models," PLoS One, 2025.

[20] Lee, S., Park, J. and Kim, H., "Feature selection for crowdfunding fraud detection," IEEE Access, 2025.

[21] Guyon, I., Weston, J., Barnhill, S. and Vapnik, V., "Gene selection for cancer classification using support vector machines," Machine Learning, 2002.

[22] Priscilla, C. V. and Prabha, D. P., "A two-phase feature selection technique using mutual information and XGB-RFE for credit card fraud detection," International Journal of Advanced Technology and Engineering Exploration, vol. 8, no. 85, pp. 1656–1668, 2021, doi: 10.19101/IJATEE.2021.874615.

[23] Darst, B. F., Malecki, K. C. and Engelman, C. D., "Using recursive feature elimination in random forest to account for correlated variables in high dimensional data," BMC Genetics, 2018.

[24] Wang, H., Liang, Q., Hancock, J. T. and Khoshgoftaar, T. M., "Feature selection strategies: a comparative analysis of SHAP-value and importance-based methods," Journal of Big Data, 2024.

[25] Tibshirani, R., "Regression shrinkage and selection via the lasso," Journal of the Royal Statistical Society: Series B, 1996.

[26] Friedman, J., Hastie, T. and Tibshirani, R., "Regularization paths for generalized linear models via coordinate descent," Journal of Statistical Software, 2010.

[27] Zou, H. and Hastie, T., "Regularization and variable selection via the elastic net," Journal of the Royal Statistical Society: Series B, 2005.

[28] El Barakaz, F., Boutkhoum, O., El Moutaouakkil, A., Rustam, F., Mehmood, A. and Choi, G. S., "Sparse feature selection with class-imbalance learning for credit card fraud detection," IEEE Access, 2021.

[29] Mniai, A., Tarik, M. and Jebari, K., "A novel framework for credit card fraud detection," IEEE Access, 2023.

[30] Ravisankar, P., Ravi, V., Raghava Rao, G. and Bose, I., "Detection of financial statement fraud and feature selection using data mining techniques," Decision Support Systems, 2011.

[31] Lee, J., Yoon, Y., Kim, J. and Kim, Y.-H., "Metaheuristic-based feature selection methods for diagnosing sarcopenia with machine learning algorithms," Biomimetics, 2024.

[32] Cox, D. R., "The regression analysis of binary sequences," Journal of the Royal Statistical Society: Series B, 1958.

[33] Rumelhart, D. E., Hinton, G. E. and Williams, R. J., "Learning representations by back-propagating errors," Nature, 1986.

[34] Rumelhart, D. E., Durbin, R., Golden, R. and Chauvin, Y., "Backpropagation: the basic theory," Backpropagation: Theory, Architectures, and Applications, Lawrence Erlbaum, 1995.

[35] Breiman, L., "Random forests," Machine Learning, 2001.

[36] Friedman, J. H., "Greedy function approximation: a gradient boosting machine," The Annals of Statistics, 2001.

[37] Chen, T. and Guestrin, C., "XGBoost: a scalable tree boosting system," Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, 2016.

[38] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q. and Liu, T.-Y., "LightGBM: a highly efficient gradient boosting decision tree," Advances in Neural Information Processing Systems 30, 2017.

[39] Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V. and Gulin, A., "CatBoost: unbiased boosting with categorical features," Advances in Neural Information Processing Systems 31, 2018.

[40] Grinsztajn, L., Oyallon, E. and Varoquaux, G., "Why do tree-based models still outperform deep learning on typical tabular data?," Advances in Neural Information Processing Systems 35, Datasets and Benchmarks Track, 2022.

[41] Demšar, J., "Statistical comparisons of classifiers over multiple data sets," Journal of Machine Learning Research, 2006.

[42] García, S. and Herrera, F., "An extension on 'statistical comparisons of classifiers over multiple data sets' for all pairwise comparisons," Journal of Machine Learning Research, 2008.

[43] Araf, I., Idri, A. and Chairi, I., "Cost-sensitive learning for imbalanced medical data: a review," Artificial Intelligence Review, 2024.

Appendix A. Exploratory Data Analysis

This appendix provides supplementary exploratory analyses referenced in Section 1.1. The missing-value profile is presented in Figure 1.1 and is not repeated here.

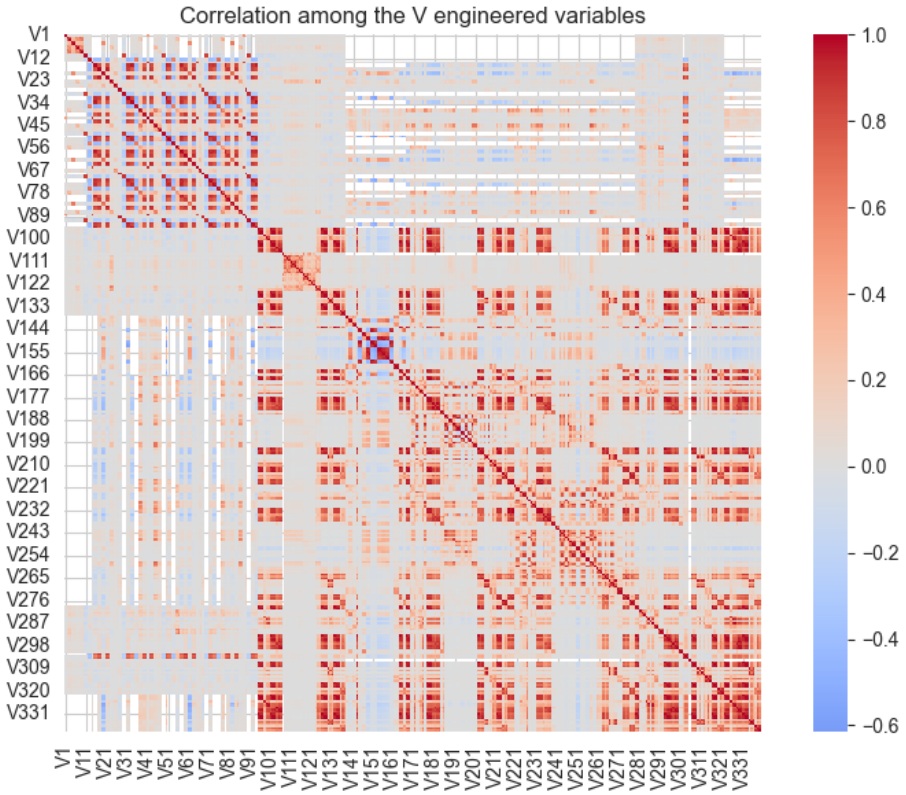


Figure A.1. Correlation among the V1–V339 engineered variables.

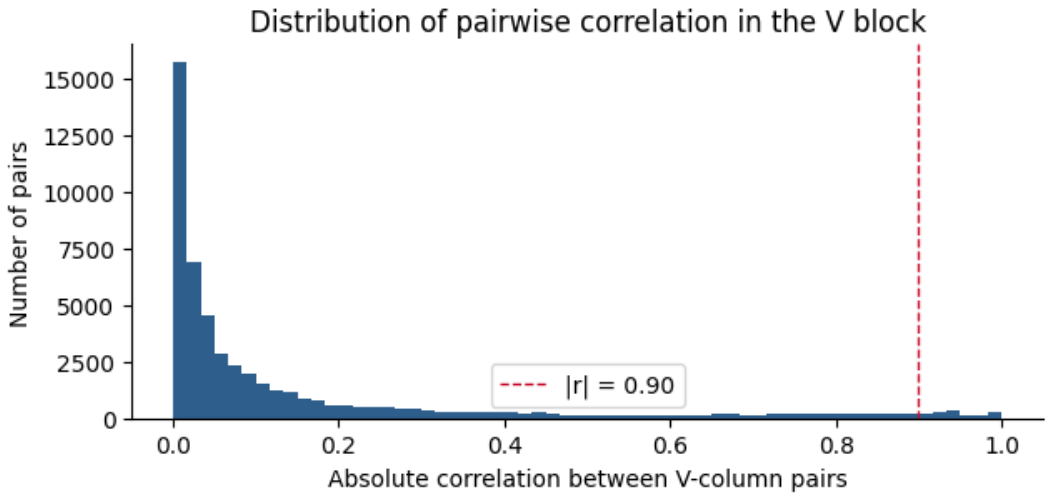


Figure A.2. Distribution of absolute pairwise correlation among the V variables (mean |r| = 0.153; 2.6% of pairs > 0.90)

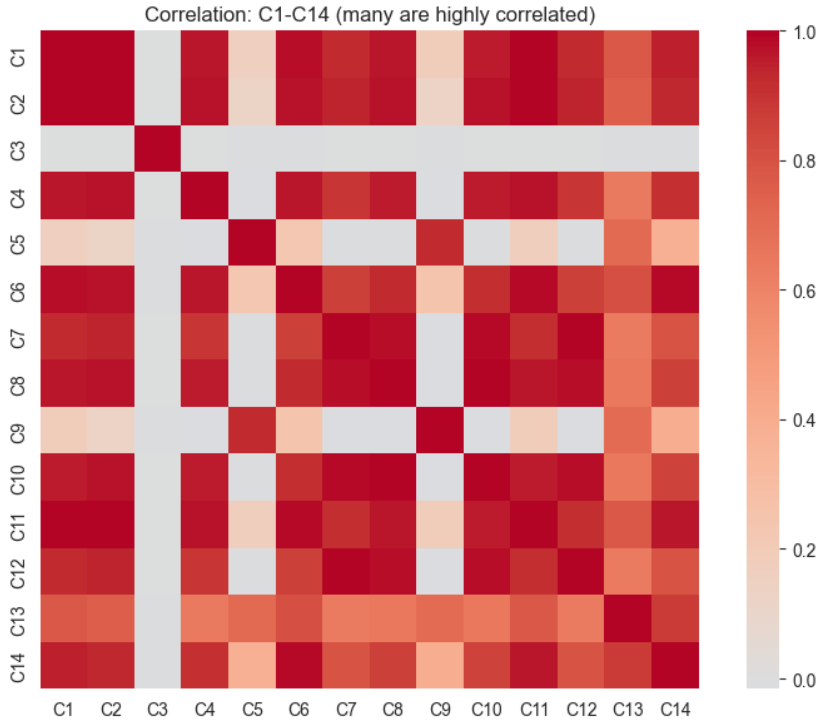


Figure A.3. Correlation among the count variables C1–C14. Most pairs are strongly positively correlated.

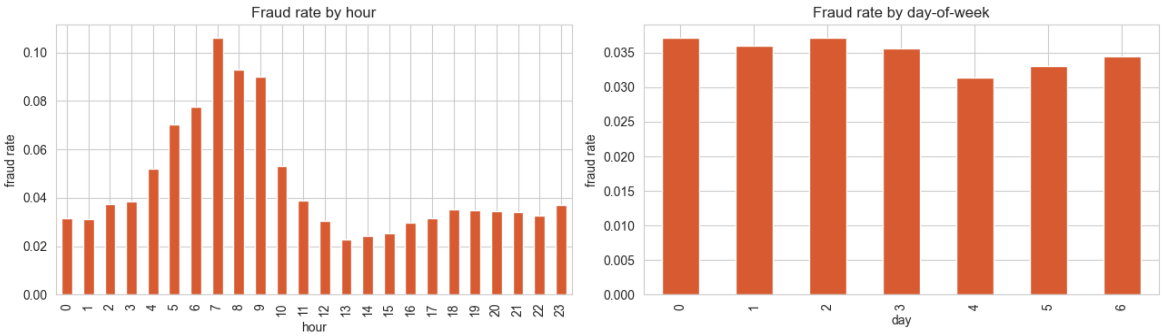


Figure A.4. Fraud rate by derived transaction hour (left) and by day of week (right).

Appendix B. Full Experimental Results

This appendix reports the complete results of the 48 experiments and the feature subsets selected by each method.

B.1 Performance of all 48 experiments

Table B.1. Held-out performance of every feature-selection and classifier combination.

Condition	Classifier	PR-AUC	ROC-AUC	MCC	F1	Precision	Recall	Train (s)	Total (min)
NO FS	LR	0.1747	0.8255	0.1951	0.1587	0.0883	0.7783	1,252.8	20.9
NO FS	RF	0.5363	0.9035	0.4423	0.3646	0.8820	0.2298	57.9	1.0
NO FS	XGB	0.5290	0.9105	0.3844	0.3606	0.2399	0.7264	44.4	0.7
NO FS	CAT	0.4924	0.9034	0.3406	0.3033	0.1894	0.7611	55.5	0.9
NO FS	LGBM	0.5256	0.9071	0.3587	0.3269	0.2094	0.7453	11.8	0.2
NO FS	MLP	0.3969	0.8308	0.2431	0.2108	0.1237	0.7104	64.7	1.1
MI	LR	0.3033	0.7630	0.1948	0.1879	0.1130	0.5571	185.3	10.1
MI	RF	0.1979	0.6853	0.1811	0.2013	0.1385	0.3679	45.1	7.8
MI	XGB	0.3857	0.7891	0.2678	0.2657	0.1757	0.5445	10.4	7.2
MI	CAT	0.3749	0.7962	0.2602	0.2523	0.1618	0.5721	36.7	7.6
MI	LGBM	0.3851	0.7925	0.2673	0.2640	0.1736	0.5507	5.3	7.1
MI	MLP	0.3568	0.7779	0.2113	0.1979	0.1186	0.5967	71.9	8.2
LASSO	LR	0.1733	0.8260	0.1904	0.1540	0.0854	0.7859	101.8	23.1
LASSO	RF	0.5106	0.8964	0.4252	0.3431	0.8801	0.2131	35.1	22.0
LASSO	XGB	0.4779	0.8949	0.3364	0.3077	0.1958	0.7175	10.1	21.6
LASSO	CAT	0.4543	0.8938	0.3118	0.2747	0.1684	0.7463	36.4	22.0
LASSO	LGBM	0.4766	0.8963	0.3307	0.2966	0.1854	0.7406	5.1	21.5
LASSO	MLP	0.3926	0.8415	0.2917	0.2698	0.1695	0.6607	63.3	22.5
RFI	LR	0.1797	0.8235	0.1841	0.1468	0.0807	0.8078	443.7	14.2
RFI	RF	0.5331	0.9060	0.4201	0.3354	0.8843	0.2069	55.8	7.7
RFI	XGB	0.5175	0.9078	0.3781	0.3554	0.2363	0.7168	20.7	7,1
RFI	CAT	0.4857	0.9039	0.3376	0.3012	0.1881	0.7557	86.4	8.2
RFI	LGBM	0.5105	0.9102	0.3654	0.3345	0.2156	0.7456	13.2	7.0
RFI	MLP	0.3588	0.8379	0.2882	0.2714	0.1728	0.6326	133.7	9.0
RFE	LR	0.1669	0.8171	0.1839	0.1499	0.0830	0.7783	424.0	13.8
RFE	RF	0.5523	0.9082	0.4509	0.3766	0.8791	0.2397	47.1	7.5
RFE	XGB	0.5161	0.9112	0.3818	0.3572	0.2366	0.7281	14.8	6.9

Condition	Classifier	PR-AUC	ROC-AUC	MCC	F1	Precision	Recall	Train (s)	Total (min)
RFE	CAT	0.4973	0.9057	0.3401	0.3023	0.1884	0.7633	50.6	7.5
RFE	LGBM	0.5084	0.9084	0.3523	0.3183	0.2018	0.7525	7.8	6.8
RFE	MLP	0.3932	0.8342	0.2807	0.2615	0.1642	0.6417	202.3	10.1
GA	LR	0.1588	0.8170	0.2198	0.1943	0.1135	0.6735	251.4	151.8
GA	RF	0.5251	0.9000	0.4602	0.3961	0.8515	0.2581	41.5	148.3
GA	XGB	0.5027	0.9015	0.3701	0.3496	0.2328	0.7013	12.1	147.8
GA	CAT	0.4748	0.8980	0.3354	0.3023	0.1900	0.7384	52.6	148.5
GA	LGBM	0.4990	0.9016	0.3569	0.3298	0.2137	0.7224	6.5	147.7
GA	MLP	0.3617	0.8322	0.2821	0.2616	0.1638	0.6494	109.6	149.4
PSO	LR	0.1702	0.8169	0.2207	0.1944	0.1134	0.6786	280.2	120.4
PSO	RF	0.5250	0.8992	0.4642	0.3947	0.8764	0.2547	41.7	116.4
PSO	XGB	0.5036	0.9043	0.3659	0.3426	0.2256	0.7114	12.4	115.9
PSO	CAT	0.4718	0.8973	0.3341	0.2998	0.1878	0.7434	52.3	116.6
PSO	LGBM	0.4939	0.9026	0.3561	0.3264	0.2099	0.7343	6.4	115.8
PSO	MLP	0.4100	0.8361	0.2844	0.2661	0.1680	0.6388	76.7	117.0
GWO	LR	0.1594	0.8093	0.2030	0.1798	0.1039	0.6671	310.4	117.0
GWO	RF	0.4507	0.8813	0.3891	0.3504	0.6784	0.2362	36.2	112.4
GWO	XGB	0.4816	0.8900	0.3562	0.3349	0.2206	0.6951	13.4	112.0
GWO	CAT	0.4530	0.8853	0.3199	0.2894	0.1812	0.7175	36.9	112.4
GWO	LGBM	0.4834	0.8934	0.3449	0.3186	0.2052	0.7114	5.6	111.9
GWO	MLP	0.3772	0.8185	0.2568	0.2380	0.1466	0.6316	55.6	112.7

Note. All metrics other than PR-AUC and ROC-AUC are computed at the fixed probability threshold of 0.50. Train (s) is the wall-clock time for fitting one classifier on one feature subset and excludes the feature-selection search. Total (min) adds the search time of the condition (Table 4.10), giving the time needed to produce that single model; because each search is shared by six classifiers, this column should not be summed.

B.2 Confusion-matrix counts

Table B.2. Confusion-matrix counts for every combination at threshold 0.50.

Condition	Classifier	TP	FN	FP	TN
NO FS	LR	3,163	901	32,645	81,399
NO FS	RF	934	3,130	125	113,919
NO FS	XGB	2,952	1,112	9,355	104,689
NO FS	CAT	3,093	971	13,236	100,808

Condition	Classifier	TP	FN	FP	TN
NO FS	LGBM	3,029	1,035	11,436	102,608
NO FS	MLP	2,887	1,177	20,445	93,599
MI	LR	2,264	1,800	17,775	96,269
MI	RF	1,495	2,569	9,298	104,746
MI	XGB	2,213	1,851	10,384	103,660
MI	CAT	2,325	1,739	12,041	102,003
MI	LGBM	2,238	1,826	10,650	103,394
MI	MLP	2,425	1,639	18,024	96,020
LASSO	LR	3,194	870	34,219	79,825
LASSO	RF	866	3,198	118	113,926
LASSO	XGB	2,916	1,148	11,975	102,069
LASSO	CAT	3,033	1,031	14,982	99,062
LASSO	LGBM	3,010	1,054	13,221	100,823
LASSO	MLP	2,685	1,379	13,153	100,891
RFI	LR	3,283	781	37,391	76,653
RFI	RF	841	3,223	110	113,934
RFI	XGB	2,913	1,151	9,414	104,630
RFI	CAT	3,071	993	13,259	100,785
RFI	LGBM	3,030	1,034	11,021	103,023
RFI	MLP	2,571	1,493	12,309	101,735
RFE	LR	3,163	901	34,961	79,083
RFE	RF	974	3,090	134	113,910
RFE	XGB	2,959	1,105	9,545	104,499
RFE	CAT	3,102	962	13,360	100,684
RFE	LGBM	3,058	1,006	12,095	101,949
RFE	MLP	2,608	1,456	13,275	100,769
GA	LR	2,737	1,327	21,369	92,675
GA	RF	1,049	3,015	183	113,861
GA	XGB	2,850	1,214	9,392	104,652
GA	CAT	3,001	1,063	12,791	101,253
GA	LGBM	2,936	1,128	10,803	103,241

Condition	Classifier	TP	FN	FP	TN
GA	MLP	2,639	1,425	13,475	100,569
PSO	LR	2,758	1,306	21,554	92,490
PSO	RF	1,035	3,029	146	113,898
PSO	XGB	2,891	1,173	9,923	104,121
PSO	CAT	3,021	1,043	13,069	100,975
PSO	LGBM	2,984	1,080	11,235	102,809
PSO	MLP	2,596	1,468	12,855	101,189
GWO	LR	2,711	1,353	23,377	90,667
GWO	RF	960	3,104	455	113,589
GWO	XGB	2,825	1,239	9,982	104,062
GWO	CAT	2,916	1,148	13,174	100,870
GWO	LGBM	2,891	1,173	11,195	102,849
GWO	MLP	2,567	1,497	14,941	99,103

Note. The test partition contains 118,108 transactions, of which 4,064 are fraudulent. Test-set results at the validation-selected thresholds for the configurations included in the supplementary threshold analysis are reported in Table 4.4.

B.3 The selected feature subsets

Each method selected 120 of the 433 predictors. One predictor, card6, was selected by all seven methods; 377 predictors were selected by at least one. The subsets are listed below in the order the selection procedure returned them.

MI V251, addr2, V247, V245, V250, V109, V256, V198, id_23, V157, V156, V248, id_25, V255, V149, V23, V193, V199, V8, V107, V155, V123, V176, V148, V67, V114, V87, V88, V154, V190, V240, V153, V189, id_22, V56, V47, V243, V86, V3, V66, V26, V170, id_27, id_17, V110, id_36, V201, V1, V41, V305, V244, id_24, id_11, V191, id_13, V252, V7, V241, V260, V254, V195, V257, V122, V242, V112, V186, id_21, V25, V115, V197, V200, V62, V228, V14, id_26, V83, V188, V20, V108, V61, V19, V55, V261, card6, D9, V82, V36, V290, V118, V38, V35, V262, V45, V113, V194, id_07, V120, V196, V53, V12, V258, V111, V44, V246, id_08, V6, id_12, V2, ProductCD, V124, V5, id_37, id_19, V37, V46, id_15, card3, V249, id_35, V9

LASSO TransactionDT, TransactionAmt, card1, card3, card4, card6, dist1, P_emaildomain, C2, C3, D2, D3, D5, D8, D11, M3, M4, M5, M6, M9, V3, V6, V13, V19, V25, V29, V30, V34, V44, V45, V47, V48, V52, V54, V55, V59, V62, V63, V67, V69, V70, V76, V79, V82, V83, V87, V94, V95, V97, V98, V101, V108, V111, V115, V123, V125, V133, V139, V141, V142, V147, V149, V150, V156, V158, V159, V160, V164, V165, V169, V173, V178, V180, V189, V200, V201, V223, V239, V242, V244, V246, V247, V252, V257, V261, V262, V264, V267, V268, V271, V274, V279, V281, V283, V284, V285, V286, V288, V296, V298, V303, V312, V318, V326, V327, V328, id_01, id_02, id_03, id_04, id_05, id_06, id_09, id_14, id_17, id_22, id_26, id_32, id_38, DeviceType

RFI C13, TransactionAmt, C14, TransactionDT, card1, card2, addr1, C1, C8, D2, D15, TransactionHour, V70, C5, D1, V317, C11, C2, card6, V294, V90, R_emaildomain, V69, card5, P_emaildomain, V307, V91, V264, V308, D10, D4, C4, V258, C6, D3, C10, V283, V30, dist1, V218, V265, V318, D5, V310, V29, D11, M4, V257, V306, id_02, V219, C12, V127, V133, V312, V282, D8, V295, ProductCD, V128, V280, V315, C9, V314, V274, C7, V130, V313, card3, V263, V102, card4, V168, V204, id_31, V285, V201, M5, M6, V246, V233, V134, DeviceInfo, id_20, V49, V279, id_17, V96, V48, id_19, V199, V131, V217, V126, id_01, V200, V203, V99, V232, V275, M3, V103, V54, V62, id_06, V243, V281, M8, V53, V97, V171, V316, V34, V309, M9, V293, V83, V291, V320, V76

RFE TransactionDT, TransactionAmt, ProductCD, card1, card2, card3, card4, card5, card6, addr1, dist1, P_emaildomain, R_emaildomain, C1, C2, C3, C4, C5, C6, C8, C9, C10, C11, C12, C13, C14, D1, D2, D3, D4, D5, D6, D8, D10, D11, D15, M2, M3, M4, M5, M6, M9, V6, V12, V13, V19, V20, V30, V37, V38, V45, V49, V53, V54, V55, V56, V61, V62, V64, V67, V69, V70, V72, V75, V76, V82, V83, V87, V90, V91, V96, V99, V102, V130, V131, V133, V136, V149, V152, V162, V165, V173, V177, V187, V197, V201, V217, V258, V261, V281, V282, V283, V285, V293, V294, V296, V307, V308, V309, V310, V311, V312, V313, V314, V315, V317, V320, V324, V333, V335, V339, id_01, id_03, id_06, id_09, id_17, id_30, id_34, DeviceType, TransactionHour

GA TransactionAmt, card1, card2, card3, card5, card6, addr1, dist1, R_emaildomain, C2, C7, C8, C9, C11, C13, D1, D4, D12, M2, M4, M5, M6, M9, V5, V15, V22, V24, V32, V33, V35, V39, V44, V45, V47, V52, V55, V56, V62, V64, V72, V73, V74, V79, V80, V90, V106, V112, V116, V121, V125, V127, V136, V142, V144, V147, V152, V154, V159, V163, V165, V166, V168, V179, V183, V191, V195, V196, V208, V216, V226, V228, V232, V238, V241, V242, V244, V248, V251, V253, V255, V256, V257, V259, V260, V263, V264, V268, V269, V274, V275, V276, V279, V280, V281, V282, V286, V288, V294, V297, V300, V302, V308, V310, V315, V316, V320, V325, V328, V333, id_01, id_06, id_07, id_10, id_22, id_24, id_26, id_28, id_29, id_30, id_36

PSO TransactionAmt, card1, card2, card3, card6, addr1, P_emaildomain, C2, C3, C4, C9, C10, C13, D2, D4, D8, D11, D12, D15, M1, M4, M6, M7, M8, M9, V1, V3, V5, V6, V10, V14, V16, V27, V30, V31, V34, V35, V37, V44, V52, V54, V57, V62, V63, V65, V66, V83, V90, V95, V105, V110, V117, V118, V123, V129, V138, V151, V153, V156, V157, V164, V166, V168, V170, V173, V175, V176, V178, V183, V184, V185, V189, V193, V216, V218, V219, V220, V221, V222, V224, V229, V234, V239, V248, V251, V254, V257, V261, V267, V271, V275, V277, V278, V281, V282, V289, V290, V291, V292, V295, V296, V299, V301, V303, V306, V317, V326, V333, V335, V337, id_06, id_09, id_11, id_14, id_16, id_20, id_28, id_32, id_33, id_37

GWO ProductCD, card1, card2, card4, card6, addr1, dist1, C4, C7, C9, C10, C11, C13, D5, D9, D10, D12, D15, M2, M4, M9, V2, V4, V7, V11, V12, V17, V22, V23, V29, V32, V33, V34, V38, V42, V44, V46, V47, V49, V50, V51, V59, V61, V63, V64, V65, V66, V71, V76, V78, V81, V86, V90, V98, V111, V117, V121, V122, V125, V139, V154, V155, V164, V165, V169, V173, V174, V183, V187, V188, V191, V192, V194, V195, V198, V205, V207, V212, V219, V225, V230, V231, V232, V233, V243, V248, V252, V255, V258, V261, V265, V273, V274, V281, V284, V286, V287, V290, V292, V295, V297, V298, V299, V300, V310, V314, V318, V319, V322, V325, V329, V331, V336, id_11, id_19, id_25, id_29, id_31, id_32, id_38

B.4 Validation-based threshold selection

Table B.4. Validation-set metrics across decision thresholds for the six configurations.

Configuration	Threshold	Precision	Recall	F1	F2	MCC
RFE + RF	**0.10 (F2*)**	**0.444**	**0.655**	**0.529**	**0.598**	**0.517**
	0.20 (MCC*)	**0.750**	**0.489**	**0.592**	**0.526**	**0.593**
	0.30	0.845	0.401	0.544	0.448	0.572
	0.40	0.896	0.338	0.491	0.386	0.541
	0.50	0.919	0.279	0.428	0.324	0.497
	0.60	0.919	0.198	0.326	0.235	0.418
	0.70	0.947	0.120	0.212	0.145	0.330
	0.80	0.983	0.062	0.116	0.076	0.241
	0.90	1.000	0.006	0.012	0.008	0.077
NO FS + RF	**0.09 (F2*)**	**0.386**	**0.675**	**0.491**	**0.587**	**0.484**
	0.10	0.432	0.644	0.517	0.587	0.504
	0.19 (MCC*)	**0.716**	**0.477**	**0.573**	**0.511**	**0.571**
	0.20	0.735	0.463	0.568	0.500	0.570
	0.30	0.847	0.367	0.512	0.414	0.547
	0.40	0.888	0.299	0.447	0.345	0.506
	0.50	0.905	0.236	0.375	0.277	0.454
	0.60	0.914	0.176	0.295	0.210	0.393
	0.70	0.923	0.117	0.208	0.142	0.322
	0.80	0.945	0.056	0.105	0.069	0.224

Configuration	Threshold	Precision	Recall	F1	F2	MCC
	0.90	1.000	0.018	0.035	0.022	0.131
RFE + XGB	0.10	0.074	0.960	0.137	0.282	0.182
	0.20	0.113	0.905	0.201	0.377	0.257
	0.30	0.160	0.842	0.268	0.454	0.317
	0.40	0.213	0.779	0.335	0.509	0.367
	0.50	0.279	0.718	0.401	0.546	0.413
	0.60	0.361	0.661	0.467	0.567	0.461
	0.62 (F2*)	**0.382**	**0.648**	**0.481**	**0.569**	**0.472**
	0.70	0.471	0.592	0.525	0.563	0.506
	0.80	0.616	0.511	0.559	0.529	0.545
	0.87 (MCC*)	**0.748**	**0.440**	**0.554**	**0.479**	**0.561**
	0.90	0.793	0.404	0.536	0.448	0.555
NO FS + XGB	0.10	0.071	0.967	0.133	0.275	0.176
	0.20	0.110	0.910	0.196	0.370	0.252
	0.30	0.157	0.853	0.266	0.453	0.316
	0.40	0.212	0.792	0.334	0.511	0.368
	0.50	0.278	0.737	0.404	0.554	0.419
	0.60	0.357	0.675	0.467	0.573	0.464
	0.65 (F2*)	**0.408**	**0.644**	**0.499**	**0.577**	**0.488**
	0.70	0.465	0.607	0.527	0.572	0.510
	0.80	0.609	0.519	0.560	0.535	0.546
	0.87 (MCC*)	**0.745**	**0.441**	**0.554**	**0.480**	**0.560**
	0.90	0.802	0.406	0.539	0.450	0.559
NO FS + LGBM	0.10	0.061	0.980	0.115	0.244	0.146
	0.20	0.090	0.940	0.165	0.327	0.219
	0.30	0.131	0.893	0.228	0.412	0.285
	0.40	0.179	0.834	0.295	0.482	0.341
	0.50	0.237	0.769	0.362	0.531	0.389
	0.60	0.311	0.704	0.432	0.562	0.437
	0.69 (F2*)	**0.408**	**0.641**	**0.499**	**0.575**	**0.487**
	0.70	0.419	0.634	0.504	0.575	0.491

Configuration	Threshold	Precision	Recall	F1	F2	MCC
	0.80	0.580	0.539	0.559	0.547	0.542
	0.88 (MCC*)	**0.746**	**0.447**	**0.559**	**0.486**	**0.565**
	0.90	0.789	0.416	0.545	0.460	0.562
MI + LR	0.10	0.039	0.998	0.076	0.170	0.005
	0.20	0.042	0.970	0.081	0.180	0.046
	0.30	0.052	0.877	0.098	0.210	0.092
	0.40	0.091	0.725	0.161	0.302	0.179
	0.50	0.127	0.629	0.212	0.352	0.222
	0.60	0.208	0.494	0.293	0.387	0.278
	0.64 (F2*)	**0.246**	**0.462**	**0.321**	**0.393**	**0.300**
	0.70	0.325	0.404	0.360	0.385	0.333
	0.80	0.492	0.342	0.404	0.364	0.391
	0.90	0.695	0.274	0.393	0.312	0.423
	0.91 (MCC*)	**0.715**	**0.266**	**0.388**	**0.305**	**0.424**

Note. Values were computed on the inner chronological validation set (94,487 transactions). Rows marked MCC and F2* are the thresholds selected by each rule; these thresholds were then applied once to the test set (Table 4.4).*

B.5 Accuracy of the 48 experiments

Table B.5. Test-set accuracy at threshold 0.50 for the 48 feature-selection–classifier combinations.

Condition	LR	RF	XGB	LGBM	CAT	MLP
NO FS	0.7160	0.9724	0.9114	0.8944	0.8797	0.8169
RFE	0.6964	0.9727	0.9098	0.8891	0.8787	0.8753
RFI	0.6768	0.9718	0.9105	0.8979	0.8793	0.8831
LASSO	0.7029	0.9719	0.8889	0.8791	0.8644	0.8770
MI	0.8343	0.8995	0.8964	0.8944	0.8833	0.8335
GA	0.8078	0.9729	0.9102	0.8990	0.8827	0.8738
PSO	0.8064	0.9731	0.9061	0.8957	0.8805	0.8787
GWO	0.7906	0.9699	0.9050	0.8953	0.8787	0.8608
Mean	**0.7539**	**0.9630**	**0.9048**	**0.8931**	**0.8784**	**0.8624**

Note. Accuracy was computed at the default threshold of 0.50 on the test set, where 3.44% of transactions are fraudulent.

B.6 Native missing-value and categorical handling

Table B.6. PR-AUC and ROC-AUC of the boosting classifiers (and RFE + RF) with the original preprocessing and with native handling of missing values and categorical variables.

Condition	Classifier	PR-AUC (original)	PR-AUC (native)	ROC-AUC (original)	ROC-AUC (native)
NO FS	XGB	0.5290	0.5249	0.9105	0.9044
NO FS	LGBM	0.5256	0.5254	0.9071	0.9060
NO FS	CAT	0.4924	0.4945	0.9034	0.9033
RFE	RF	0.5523	0.5473	0.9082	0.9071
RFE	XGB	0.5161	0.5196	0.9112	0.9109
RFE	LGBM	0.5084	0.5197	0.9084	0.9111
RFE	CAT	0.4973	0.4931	0.9057	0.9045

Note. Native handling left numerical missing values unimputed and used the models' built-in categorical support (Random Forest used native missing values with the original categorical encoding). Class weighting and hyperparameters were identical to the main experiments. The RFE subset was the original 120-feature subset and was not reselected.

Appendix C. Shaffer Post-hoc Comparisons

This appendix reports the complete pairwise comparisons following the Friedman tests in Section 4.4. P-values were adjusted using Shaffer's static step-down procedure to control the family-wise error rate. Tables C.1 and C.2 report the comparisons on PR-AUC, and Tables C.3 and C.4 the same comparisons on ROC-AUC.

C.1 Feature-selection conditions, PR-AUC

Eight feature-selection conditions produced 28 pairwise comparisons.

Table C.1. All 28 pairwise comparisons among the eight feature-selection conditions, on PR-AUC.

Comparison	Rank difference	z	Unadjusted p	Shaffer-adjusted p	Significant
NO FS vs MI	-5.000	-3.536	0.0004	0.0114	Yes
NO FS vs GWO	-4.500	-3.182	0.0015	0.0307	Yes
MI vs RFE	+4.000	+2.828	0.0047	0.0982	No
NO FS vs LASSO	-3.833	-2.711	0.0067	0.1411	No
MI vs RFI	+3.667	+2.593	0.0095	0.2000	No
RFE vs GWO	-3.500	-2.475	0.0133	0.2799	No
NO FS vs GA	-3.333	-2.357	0.0184	0.3869	No
RFI vs GWO	-3.167	-2.239	0.0251	0.5280	No
LASSO vs RFE	+2.833	+2.003	0.0451	0.7220	No
MI vs PSO	+2.667	+1.886	0.0593	0.9495	No
LASSO vs RFI	+2.500	+1.768	0.0771	1.0000	No
NO FS vs PSO	-2.333	-1.650	0.0990	1.0000	No
RFE vs GA	-2.333	-1.650	0.0990	1.0000	No
PSO vs GWO	-2.167	-1.532	0.1255	1.0000	No
RFI vs GA	-2.000	-1.414	0.1573	1.0000	No
MI vs GA	+1.667	+1.179	0.2386	1.0000	No
LASSO vs PSO	+1.500	+1.061	0.2888	1.0000	No
RFE vs PSO	-1.333	-0.943	0.3458	1.0000	No
NO FS vs RFI	-1.333	-0.943	0.3458	1.0000	No
MI vs LASSO	+1.167	+0.825	0.4094	1.0000	No
GA vs GWO	-1.167	-0.825	0.4094	1.0000	No
RFI vs PSO	-1.000	-0.707	0.4795	1.0000	No
NO FS vs RFE	-1.000	-0.707	0.4795	1.0000	No
GA vs PSO	+1.000	+0.707	0.4795	1.0000	No

Comparison	Rank difference	z	Unadjusted p	Shaffer-adjusted p	Significant
LASSO vs GWO	-0.667	-0.471	0.6374	1.0000	No
MI vs GWO	+0.500	+0.354	0.7237	1.0000	No
LASSO vs GA	+0.500	+0.354	0.7237	1.0000	No
RFI vs RFE	+0.333	+0.236	0.8137	1.0000	No

Note. A negative rank difference means the first condition has the better (lower) mean rank. Two comparisons remain significant at α = 0.05 after adjustment: NO FS against MI and NO FS against GWO. With only six blocks, the test has limited power to separate conditions whose mean ranks differ by about one.

C.2 Classifiers, PR-AUC

Six classifiers produced 15 pairwise comparisons.

Table C.2. All 15 pairwise comparisons among the six classifiers, on PR-AUC.

Comparison	Rank difference	z	Unadjusted p	Shaffer-adjusted p	Significant
LR vs XGB	+4.000	+4.276	0.0000	0.0003	Yes
LR vs RF	+3.875	+4.143	0.0000	0.0003	Yes
LR vs LGBM	+3.250	+3.474	0.0005	0.0051	Yes
XGB vs MLP	-3.000	-3.207	0.0013	0.0134	Yes
RF vs MLP	-2.875	-3.074	0.0021	0.0212	Yes
LGBM vs MLP	-2.250	-2.405	0.0162	0.1616	No
LR vs CAT	+2.125	+2.272	0.0231	0.1617	No
XGB vs CAT	-1.875	-2.004	0.0450	0.3151	No
RF vs CAT	-1.750	-1.871	0.0614	0.4296	No
CAT vs MLP	-1.125	-1.203	0.2291	1.0000	No
CAT vs LGBM	+1.125	+1.203	0.2291	1.0000	No
LR vs MLP	+1.000	+1.069	0.2850	1.0000	No
XGB vs LGBM	-0.750	-0.802	0.4227	1.0000	No
RF vs LGBM	-0.625	-0.668	0.5040	1.0000	No
RF vs XGB	+0.125	+0.134	0.8937	1.0000	No

Note. A negative rank difference means the first classifier has the better (lower) mean rank. Five comparisons remain significant after adjustment. All of them contrast a tree ensemble with either Logistic Regression or the MLP; no pair among XGB, RF and LGBM is separated.

C.3 Feature-selection conditions, ROC-AUC

Table C.3. All 28 pairwise comparisons among the eight feature-selection conditions, on ROC-AUC.

Comparison	Rank difference	z	Unadjusted p	Shaffer-adjusted p	Significant
MI vs RFE	+5.833	+4.125	0.0000	0.0010	Yes
MI vs RFI	+5.833	+4.125	0.0000	0.0010	Yes
NO FS vs MI	-4.833	-3.418	0.0006	0.0133	Yes
RFI vs GWO	-4.833	-3.418	0.0006	0.0133	Yes
RFE vs GWO	-4.833	-3.418	0.0006	0.0133	Yes
NO FS vs GWO	-3.833	-2.711	0.0067	0.1411	No
MI vs LASSO	+3.667	+2.593	0.0095	0.2000	No
MI vs PSO	+3.500	+2.475	0.0133	0.2799	No
MI vs GA	+3.333	+2.357	0.0184	0.2948	No
LASSO vs GWO	-2.667	-1.886	0.0593	0.9495	No
RFE vs GA	-2.500	-1.768	0.0771	1.0000	No
RFI vs GA	-2.500	-1.768	0.0771	1.0000	No
PSO vs GWO	-2.500	-1.768	0.0771	1.0000	No
RFI vs PSO	-2.333	-1.650	0.0990	1.0000	No
RFE vs PSO	-2.333	-1.650	0.0990	1.0000	No
GA vs GWO	-2.333	-1.650	0.0990	1.0000	No
LASSO vs RFI	+2.167	+1.532	0.1255	1.0000	No
LASSO vs RFE	+2.167	+1.532	0.1255	1.0000	No
NO FS vs GA	-1.500	-1.061	0.2888	1.0000	No
NO FS vs PSO	-1.333	-0.943	0.3458	1.0000	No
NO FS vs LASSO	-1.167	-0.825	0.4094	1.0000	No
MI vs GWO	+1.000	+0.707	0.4795	1.0000	No
NO FS vs RFE	+1.000	+0.707	0.4795	1.0000	No
NO FS vs RFI	+1.000	+0.707	0.4795	1.0000	No
LASSO vs GA	-0.333	-0.236	0.8137	1.0000	No
LASSO vs PSO	-0.167	-0.118	0.9062	1.0000	No
GA vs PSO	+0.167	+0.118	0.9062	1.0000	No
RFI vs RFE	+0.000	+0.000	1.0000	1.0000	No

Note. A positive rank difference means the first condition has the worse (higher) mean rank. Five comparisons remain significant at $\alpha = 0.05$ after adjustment: RFE and RFI against MI and GWO, and NO FS against MI.

C.4 Classifiers, ROC-AUC

Table C.4. All 15 pairwise comparisons among the six classifiers, on ROC-AUC.

Comparison	Rank difference	z	Unadjusted p	Shaffer-adjusted p	Significant
LR vs LGBM	+4.250	+4.543	0.0000	0.0001	Yes
LR vs XGB	+4.000	+4.276	0.0000	0.0002	Yes
LGBM vs MLP	-3.250	-3.474	0.0005	0.0051	Yes
XGB vs MLP	-3.000	-3.207	0.0013	0.0134	Yes
LR vs RF	+2.625	+2.806	0.0050	0.0501	No
LR vs CAT	+2.375	+2.539	0.0111	0.1112	No
CAT vs LGBM	+1.875	+2.004	0.0450	0.3151	No
RF vs LGBM	+1.625	+1.737	0.0824	0.5765	No
XGB vs CAT	-1.625	-1.737	0.0824	0.5765	No
RF vs MLP	-1.625	-1.737	0.0824	0.5765	No
RF vs XGB	+1.375	+1.470	0.1416	0.5765	No
CAT vs MLP	-1.375	-1.470	0.1416	0.5765	No
LR vs MLP	+1.000	+1.069	0.2850	0.8551	No
RF vs CAT	-0.250	-0.267	0.7893	1.0000	No
XGB vs LGBM	+0.250	+0.267	0.7893	1.0000	No

Note. A positive rank difference means the first classifier has the worse (higher) mean rank. Four comparisons remain significant after adjustment, each contrasting Logistic Regression or the MLP with LightGBM or XGBoost.

Appendix D. Preprocessing Implementation

The code below is the preprocessing pipeline described in Section 3.2.2

Step 1 Load and merge

```
import numpy as np, pandas as pd

tx = pd.read_csv('data/train_transaction.csv')
id = pd.read_csv('data/train_identity.csv')
df = tx.merge(id, how='left', on='TransactionID')
del tx, id
df = df.drop(columns=['TransactionID'])
```

Step 2 Derive the transaction hour

TransactionDT is a seconds offset from a fixed reference point, so the hour of day is recovered by integer division.

```
df["TransactionHour"] = ((df["TransactionDT"] // 3600) % 24)


# Identify feature types
TARGET= "isFraud"
num_cols= [c for c in df.select_dtypes(include=["number"]).columns
           if c not in (TARGET, "TransactionDT")]
cat_cols= list(df.select_dtypes(include=["object", "category"]).columns)
```

Step 3 Chronological 80/20 split

Rows are sorted by ascending TransactionDT and divided sequentially

```
df = df.sort_values("TransactionDT").reset_index(drop=True)
cut= int(len(df) * 0.80)
train = df.iloc[:cut].copy()   # earlier (80%)
test = df.iloc[cut:].copy()   # later   (20%)
```

Step 4 Numeric imputation

Medians are computed on the training partition and applied to both partitions.

```
train_medians = train[num_cols].median()
train[num_cols] = train[num_cols].fillna(train_medians)
test[num_cols]  = test[num_cols].fillna(train_medians)
```

Step 5 Encode categorical

```
encoders = {}
for col in cat_cols:
    tr_na, te_na = train[col].isna(), test[col].isna()
    tr_s,  te_s  = train[col].astype(str), test[col].astype(str)

    levels  = sorted(tr_s[~tr_na].unique())
    mapping = {lvl: i for i, lvl in enumerate(levels)}

    train[col] = tr_s.map(mapping).where(~tr_na, -1).astype("int32")

    te_code = te_s.map(mapping).where(~te_na, -1)   # missing -> -1
    test[col] = te_code.fillna(-2).astype("int32")  # unseen  -> -2
    encoders[col] = mapping
```

Step 6 Separate features and target

```
X_train = train.drop(columns=[TARGET]).copy()
y_train = train[TARGET].copy()
X_test = test.drop(columns=[TARGET]).copy()
y_test = test[TARGET].copy()
```